Maze-Solver

0.8.1

Generated by Doxygen 1.18.0

1 Maze-Solver 0.8.1 1
1.1 Table of Contents . . . 1
1.2 Overview . . . 2
1.3 Features . . . 2
1.4 Requirements . . . 3
1.5 Quick Start . . . 3
1.6 How to Use . . . 5
1.7 Maze File Format . . . 6
1.7.1 Weighted map format . . . 7
1.8 Included Maze Files . . . 8
1.9 Algorithms . . . 8
1.10 Rendering and Performance . . . 9
1.11 Project Structure . . . 9
1.12 API and Extension Notes . . . 10
1.13 Launch From Release JAR . . . 11
1.14 Troubleshooting . . . 11
1.15 Development Notes . . . 12
1.15.1 Developer Notes: experiment execution . . . 12
1.16 License . . . 13

2 Directory Hierarchy 15
2.1 Directories . . . 15

3 Namespace Index 17
3.1 Namespace List . . . 17

4 Hierarchical Index 19
4.1 Class Hierarchy . . . 19

5 Class Index 21
5.1 Class List . . . 21

6 File Index 23
6.1 File List . . . 23

7 Directory Documentation 25
7.1 C:/Users/ramse/Github/Maze-Solver/src/app Directory Reference . . . 25
7.2 C:/Users/ramse/Github/Maze-Solver/src/experiments Directory Reference . . . 25
7.3 C:/Users/ramse/Github/Maze-Solver/src/maze/map Directory Reference . . . 25
7.4 C:/Users/ramse/Github/Maze-Solver/src/maze Directory Reference . . . 25
7.5 C:/Users/ramse/Github/Maze-Solver/src/maze/search Directory Reference . . . 26
7.6 C:/Users/ramse/Github/Maze-Solver/src Directory Reference . . . 26
7.7 C:/Users/ramse/Github/Maze-Solver/src/ui Directory Reference . . . 26

8 Namespace Documentation 27
8.1 Package app . . . 27

8.2 Package experiments . . . 27
8.3 Package maze.map . . . 27
8.4 Package maze.search . . . 28
8.5 Package ui . . . 28

9 Class Documentation 29
9.1 maze.search.MazeSolver.Algorithm Enum Reference . . . 29
9.1.1 Detailed Description . . . 29
9.1.2 Member Data Documentation . . . 29
9.1.2.1 A_STAR . . . 29
9.1.2.2 BFS . . . 30
9.1.2.3 DFS . . . 30
9.1.2.4 GREEDY . . . 30
9.1.2.5 UCS . . . 30
9.2 app.ApplicationPreferences Class Reference . . . 30
9.2.1 Detailed Description . . . 30
9.2.2 Member Function Documentation . . . 31
9.2.2.1 getMaxHeapGb() . . . 31
9.2.2.2 preferencesPath() . . . 31
9.2.2.3 setMaxHeapGb() . . . 31
9.3 experiments.ExperimentRunner.CancellationToken Interface Reference . . . 32
9.3.1 Detailed Description . . . 32
9.3.2 Member Function Documentation . . . 32
9.3.2.1 isCancelled() . . . 32
9.4 experiments.TestSuite.CostMode Enum Reference . . . 32
9.4.1 Detailed Description . . . 33
9.4.2 Member Data Documentation . . . 33
9.4.2.1 FROM_MAP . . . 33
9.4.2.2 RANDOM . . . 33
9.4.2.3 UNIFORM . . . 33
9.5 maze.map.Maze.Dir Enum Reference . . . 33
9.5.1 Detailed Description . . . 34
9.5.2 Constructor & Destructor Documentation . . . 34
9.5.2.1 Dir() . . . 34
9.5.3 Member Function Documentation . . . 34
9.5.3.1 idx() . . . 34
9.5.4 Member Data Documentation . . . 34
9.5.4.1 EAST . . . 34
9.5.4.2 NORTH . . . 35
9.5.4.3 SOUTH . . . 35
9.5.4.4 WEST . . . 35
9.6 experiments.TestSuite.EndpointMode Enum Reference . . . 35
9.6.1 Detailed Description . . . 35

9.6.2 Member Data Documentation . . . 35
9.6.2.1 APPROXIMATE_DISTANCE . . . 35
9.6.2.2 FIXED . . . 36
9.6.2.3 OPPOSITE_CORNERS . . . 36
9.6.2.4 RANDOM . . . 36
9.7 experiments.ExperimentRunner.ExecutionControl Interface Reference . . . 36
9.7.1 Detailed Description . . . 37
9.7.2 Member Function Documentation . . . 37
9.7.2.1 awaitIfPaused() . . . 37
9.7.2.2 isPaused() . . . 37
9.8 experiments.TestSuite.ExecutionMode Enum Reference . . . 37
9.8.1 Detailed Description . . . 37
9.8.2 Member Function Documentation . . . 38
9.8.2.1 toString() . . . 38
9.8.3 Member Data Documentation . . . 38
9.8.3.1 ADAPTIVE_PARALLEL . . . 38
9.8.3.2 FIXED_PARALLEL . . . 38
9.8.3.3 SEQUENTIAL . . . 38
9.9 experiments.ExperimentCli Class Reference . . . 38
9.9.1 Detailed Description . . . 38
9.9.2 Member Function Documentation . . . 39
9.9.2.1 main() . . . 39
9.10 experiments.ExperimentResult Class Reference . . . 39
9.10.1 Detailed Description . . . 40
9.10.2 Constructor & Destructor Documentation . . . 41
9.10.2.1 ExperimentResult() . . . 41
9.10.3 Member Function Documentation . . . 42
9.10.3.1 getActualBlockedPercent() . . . 42
9.10.3.2 getActualManhattanDistance() . . . 42
9.10.3.3 getAlgorithm() . . . 43
9.10.3.4 getBlockedCellCount() . . . 43
9.10.3.5 getColumns() . . . 43
9.10.3.6 getConfigurationId() . . . 43
9.10.3.7 getCostMode() . . . 43
9.10.3.8 getEffectiveSeed() . . . 44
9.10.3.9 getGoalColumn() . . . 44
9.10.3.10 getGoalRow() . . . 44
9.10.3.11 getIteration() . . . 44
9.10.3.12 getMapSource() . . . 44
9.10.3.13 getMaximumCost() . . . 45
9.10.3.14 getMinimumCost() . . . 45
9.10.3.15 getNodesExplored() . . . 45
9.10.3.16 getPathCost() . . . 45

9.10.3.17 getPathLength() . . . 45
9.10.3.18 getReachabilityMode() . . . 46
9.10.3.19 getRequestedBlockedPercent() . . . 46
9.10.3.20 getRequestedManhattanDistance() . . . 46
9.10.3.21 getRows() . . . 46
9.10.3.22 getRuntimeMs() . . . 46
9.10.3.23 getRuntimeNs() . . . 47
9.10.3.24 getSavedMapName() . . . 47
9.10.3.25 getSeedLabel() . . . 47
9.10.3.26 getStartColumn() . . . 47
9.10.3.27 getStartRow() . . . 47
9.10.3.28 getSuiteName() . . . 48
9.10.3.29 isGeneratedMapSolvable() . . . 48
9.10.3.30 isSolved() . . . 48
9.11 experiments.ExperimentRunner Class Reference . . . 48
9.11.1 Detailed Description . . . 49
9.11.2 Member Function Documentation . . . 50
9.11.2.1 exportMaps() . . . 50
9.11.2.2 formatSummary() . . . 50
9.11.2.3 getActiveMapWorkerCount() . . . 50
9.11.2.4 getActiveSolverWorkerCount() . . . 51
9.11.2.5 getActiveWorkerCount() . . . 51
9.11.2.6 getAdaptiveWorkerCount() . . . 51
9.11.2.7 getIntervalPeakMapWorkerCount() . . . 51
9.11.2.8 getIntervalPeakSolverWorkerCount() . . . 51
9.11.2.9 getParallelWorkerCount() . . . 52
9.11.2.10 run() [1/4] . . . 52
9.11.2.11 run() [2/4] . . . 52
9.11.2.12 run() [3/4] . . . 53
9.11.2.13 run() [4/4] . . . 53
9.11.3 Member Data Documentation . . . 54
9.11.3.1 NEVER_CANCEL . . . 54
9.12 maze.map.Maze.GeneratedExperimentMap Class Reference . . . 54
9.12.1 Detailed Description . . . 54
9.12.2 Member Function Documentation . . . 55
9.12.2.1 fromSavedMap() . . . 55
9.12.2.2 getActualBlockedPercent() . . . 55
9.12.2.3 getActualManhattanDistance() . . . 55
9.12.2.4 getBlockedCellCount() . . . 55
9.12.2.5 getEffectiveSeed() . . . 56
9.12.2.6 getGoalColumn() . . . 56
9.12.2.7 getGoalRow() . . . 56
9.12.2.8 getMaze() . . . 56

9.12.2.9 getStartColumn() . . . 56
9.12.2.10 getStartRow() . . . 57
9.12.2.11 isSolvable() . . . 57
9.13 experiments.ExperimentRunner.MapProgressListener Interface Reference . . . 57
9.13.1 Detailed Description . . . 57
9.13.2 Member Function Documentation . . . 58
9.13.2.1 onProgress() . . . 58
9.14 experiments.TestSuite.MapSource Enum Reference . . . 58
9.14.1 Detailed Description . . . 58
9.14.2 Member Data Documentation . . . 59
9.14.2.1 AUTOMATIC_SEED . . . 59
9.14.2.2 MANUAL_SEED . . . 59
9.14.2.3 SAVED_MAP . . . 59
9.15 maze.map.Maze Class Reference . . . 59
9.15.1 Detailed Description . . . 61
9.15.2 Constructor & Destructor Documentation . . . 62
9.15.2.1 Maze() . . . 62
9.15.3 Member Function Documentation . . . 62
9.15.3.1 buildNeighborTableLines() . . . 62
9.15.3.2 canMove() . . . 62
9.15.3.3 createCell() . . . 63
9.15.3.4 exportToFile() . . . 63
9.15.3.5 exportToText() . . . 63
9.15.3.6 generateExperimentMap() . . . 64
9.15.3.7 generateRandomMaze() [1/3] . . . 64
9.15.3.8 generateRandomMaze() [2/3] . . . 65
9.15.3.9 generateRandomMaze() [3/3] . . . 65
9.15.3.10 getGoal() . . . 66
9.15.3.11 getLength() . . . 66
9.15.3.12 getMoveCost() . . . 67
9.15.3.13 getNode() . . . 67
9.15.3.14 getStart() . . . 67
9.15.3.15 getWidth() . . . 68
9.15.3.16 importFromFile() . . . 68
9.15.3.17 importFromText() . . . 68
9.15.3.18 isInBounds() . . . 69
9.15.3.19 isOccupied() . . . 69
9.15.3.20 setGoal() . . . 70
9.15.3.21 setStart() . . . 70
9.15.3.22 validateNeighborConnections() . . . 70
9.15.4 Member Data Documentation . . . 70
9.15.4.1 DISABLED_MOVEMENT_COST . . . 70
9.15.4.2 MAP_EXTENSION . . . 71

9.15.4.3 MAX_MOVEMENT_COST . . . 71
9.15.4.4 MIN_MOVEMENT_COST . . . 71
9.16 maze.map.MazeFileIO Class Reference . . . 71
9.16.1 Detailed Description . . . 71
9.16.2 Member Function Documentation . . . 72
9.16.2.1 exportToFile() . . . 72
9.16.2.2 exportToText() . . . 72
9.16.2.3 importFromFile() . . . 72
9.16.2.4 importFromText() . . . 73
9.17 ui.MazeRenderer Class Reference . . . 73
9.17.1 Detailed Description . . . 74
9.17.2 Member Function Documentation . . . 74
9.17.2.1 launchFromTitleScreen() . . . 74
9.17.2.2 resetNodeColor() . . . 75
9.17.2.3 reviewExperiment() [1/2] . . . 75
9.17.2.4 reviewExperiment() [2/2] . . . 75
9.17.2.5 setNodeColor() . . . 76
9.17.2.6 shutdownApplication() . . . 76
9.18 maze.search.MazeSolver Class Reference . . . 76
9.18.1 Detailed Description . . . 77
9.18.2 Constructor & Destructor Documentation . . . 78
9.18.2.1 MazeSolver() . . . 78
9.18.3 Member Function Documentation . . . 78
9.18.3.1 getAlg() . . . 78
9.18.3.2 getCurrentPathCost() . . . 78
9.18.3.3 getFinalPath() . . . 78
9.18.3.4 getPathCost() . . . 79
9.18.3.5 getSolveTimeNs() . . . 79
9.18.3.6 getVisitedCount() . . . 79
9.18.3.7 isFinished() . . . 79
9.18.3.8 isSolved() . . . 79
9.18.3.9 resetSearch() . . . 80
9.18.3.10 setAlg() . . . 80
9.18.3.11 setMaze() . . . 80
9.18.3.12 updateSolve() . . . 80
9.19 app.MS Class Reference . . . 81
9.19.1 Detailed Description . . . 81
9.19.2 Constructor & Destructor Documentation . . . 81
9.19.2.1 MS() . . . 81
9.19.3 Member Function Documentation . . . 82
9.19.3.1 main() . . . 82
9.20 maze.map.Maze.NeighborValidationResult Class Reference . . . 82
9.20.1 Detailed Description . . . 82

9.20.2 Member Function Documentation . . . 82
9.20.2.1 getIssueCount() . . . 82
9.20.2.2 getIssues() . . . 83
9.20.2.3 isValid() . . . 83
9.21 maze.map.Maze.Node Class Reference . . . 83
9.21.1 Detailed Description . . . 84
9.21.2 Constructor & Destructor Documentation . . . 84
9.21.2.1 Node() [1/2] . . . 84
9.21.2.2 Node() [2/2] . . . 84
9.21.3 Member Function Documentation . . . 85
9.21.3.1 getMoveCost() . . . 85
9.21.3.2 getNeighbor() . . . 85
9.21.3.3 setMoveCost() . . . 85
9.21.3.4 setNeighbor() . . . 86
9.21.4 Member Data Documentation . . . 86
9.21.4.1 col . . . 86
9.21.4.2 isGoal . . . 86
9.21.4.3 neighbors . . . 86
9.21.4.4 row . . . 86
9.22 experiments.ExperimentRunner.ProgressListener Interface Reference . . . 87
9.22.1 Detailed Description . . . 87
9.22.2 Member Function Documentation . . . 87
9.22.2.1 onProgress() . . . 87
9.23 experiments.TestSuite.ReachabilityMode Enum Reference . . . 87
9.23.1 Detailed Description . . . 88
9.23.2 Member Data Documentation . . . 88
9.23.2.1 GUARANTEED_SOLVABLE . . . 88
9.23.2.2 GUARANTEED_UNSOLVABLE . . . 88
9.23.2.3 NATURAL . . . 88
9.24 maze.search.SearchStrategy Interface Reference . . . 88
9.24.1 Detailed Description . . . 88
9.24.2 Member Function Documentation . . . 89
9.24.2.1 initialize() . . . 89
9.24.2.2 step() . . . 89
9.25 experiments.ExperimentRunner.SolveProgress Class Reference . . . 89
9.25.1 Detailed Description . . . 89
9.25.2 Member Function Documentation . . . 90
9.25.2.1 getAlgorithm() . . . 90
9.25.2.2 getConfigurationId() . . . 90
9.25.2.3 getElapsedNs() . . . 90
9.25.2.4 getIteration() . . . 90
9.25.2.5 getNodesExplored() . . . 90
9.25.2.6 getPathCost() . . . 91

9.25.2.7 isFinished() . . . 91
9.26 experiments.ExperimentRunner.SolveProgressListener Interface Reference . . . 91
9.26.1 Detailed Description . . . 91
9.26.2 Member Function Documentation . . . 92
9.26.2.1 onProgress() . . . 92
9.27 experiments.ExperimentRunner.TaskPriority Enum Reference . . . 92
9.27.1 Constructor & Destructor Documentation . . . 92
9.27.1.1 TaskPriority() . . . 92
9.27.2 Member Data Documentation . . . 92
9.27.2.1 MAP . . . 92
9.27.2.2 SOLVER . . . 92
9.28 experiments.TestSuite.TestCase Class Reference . . . 93
9.28.1 Detailed Description . . . 95
9.28.2 Constructor & Destructor Documentation . . . 95
9.28.2.1 TestCase() [1/2] . . . 95
9.28.2.2 TestCase() [2/2] . . . 95
9.28.3 Member Function Documentation . . . 95
9.28.3.1 getAlgorithms() . . . 95
9.28.3.2 getBlockedPercent() . . . 95
9.28.3.3 getColumns() . . . 96
9.28.3.4 getCostMode() . . . 96
9.28.3.5 getDistanceTolerance() . . . 96
9.28.3.6 getEndpointMode() . . . 96
9.28.3.7 getGoalColumn() . . . 96
9.28.3.8 getGoalRow() . . . 97
9.28.3.9 getId() . . . 97
9.28.3.10 getIterations() . . . 97
9.28.3.11 getMapSource() . . . 97
9.28.3.12 getMaximumCost() . . . 97
9.28.3.13 getMinimumCost() . . . 98
9.28.3.14 getReachabilityMode() . . . 98
9.28.3.15 getRows() . . . 98
9.28.3.16 getSavedMapName() . . . 98
9.28.3.17 getSavedMapText() . . . 98
9.28.3.18 getSeed() . . . 99
9.28.3.19 getStartColumn() . . . 99
9.28.3.20 getStartRow() . . . 99
9.28.3.21 getTargetManhattanDistance() . . . 99
9.28.3.22 isAutomaticSeed() . . . 99
9.28.3.23 isSameCostBothDirections() . . . 100
9.28.3.24 setAlgorithms() . . . 100
9.28.3.25 setAutomaticSeed() . . . 100
9.28.3.26 setBlockedPercent() . . . 100

9.28.3.27 setColumns() . . . 101
9.28.3.28 setCostMode() . . . 101
9.28.3.29 setDistanceTolerance() . . . 101
9.28.3.30 setEndpointMode() . . . 101
9.28.3.31 setGoalColumn() . . . 102
9.28.3.32 setGoalRow() . . . 102
9.28.3.33 setId() . . . 102
9.28.3.34 setIterations() . . . 102
9.28.3.35 setMapSource() . . . 103
9.28.3.36 setMaximumCost() . . . 103
9.28.3.37 setMinimumCost() . . . 103
9.28.3.38 setReachabilityMode() . . . 103
9.28.3.39 setRows() . . . 104
9.28.3.40 setSameCostBothDirections() . . . 104
9.28.3.41 setSavedMapName() . . . 104
9.28.3.42 setSavedMapText() . . . 104
9.28.3.43 setSeed() . . . 105
9.28.3.44 setStartColumn() . . . 105
9.28.3.45 setStartRow() . . . 105
9.28.3.46 setTargetManhattanDistance() . . . 105
9.28.3.47 validate() . . . 106
9.29 experiments.TestSuite Class Reference . . . 106
9.29.1 Detailed Description . . . 107
9.29.2 Constructor & Destructor Documentation . . . 107
9.29.2.1 TestSuite() . . . 107
9.29.3 Member Function Documentation . . . 108
9.29.3.1 addTestCase() . . . 108
9.29.3.2 getExecutionMode() . . . 108
9.29.3.3 getExportDirectory() . . . 108
9.29.3.4 getName() . . . 108
9.29.3.5 getTestCases() . . . 109
9.29.3.6 isAutomaticExport() . . . 109
9.29.3.7 isParallelExecution() . . . 109
9.29.3.8 removeTestCase() . . . 109
9.29.3.9 setAutomaticExport() . . . 110
9.29.3.10 setExecutionMode() . . . 110
9.29.3.11 setExportDirectory() . . . 110
9.29.3.12 setName() . . . 110
9.29.3.13 setParallelExecution() . . . 111
9.29.3.14 validate() . . . 111
9.30 experiments.TestSuiteIO Class Reference . . . 111
9.30.1 Detailed Description . . . 112
9.30.2 Member Function Documentation . . . 112

9.30.2.1 defaultExportDirectory() . . . 112
9.30.2.2 exportCsv() . . . 112
9.30.2.3 load() . . . 113
9.30.2.4 save() . . . 113
9.30.3 Member Data Documentation . . . 113
9.30.3.1 SUITE_EXTENSION . . . 113
9.30.3.2 SUITE_FORMAT_VERSION . . . 114
9.31 ui.TestSuiteWindow Class Reference . . . 114
9.31.1 Detailed Description . . . 114
9.31.2 Member Function Documentation . . . 114
9.31.2.1 closeWindow() . . . 114
9.31.2.2 showWindow() [1/2] . . . 114
9.31.2.3 showWindow() [2/2] . . . 115
9.32 maze.search.UninformedSearch.TraversalMode Enum Reference . . . 115
9.32.1 Member Data Documentation . . . 115
9.32.1.1 BFS . . . 115
9.32.1.2 DFS . . . 115
9.33 app.Version Class Reference . . . 115
9.33.1 Detailed Description . . . 116
9.33.2 Member Data Documentation . . . 116
9.33.2.1 RELEASE . . . 116
9.34 ui.MazeRenderer.WeightDisplayState Enum Reference . . . 116
9.34.1 Detailed Description . . . 116
9.34.2 Member Data Documentation . . . 116
9.34.2.1 FINAL_PATH . . . 116
9.34.2.2 NORMAL . . . 116
9.34.2.3 TRAVERSED . . . 116

10 File Documentation 117
10.1 C:/Users/ramse/Github/Maze-Solver/README.md File Reference . . . 117
10.2 C:/Users/ramse/Github/Maze-Solver/src/app/ApplicationPreferences.java File Reference 117
10.3 C:/Users/ramse/Github/Maze-Solver/src/app/MS.java File Reference . . . 117
10.4 C:/Users/ramse/Github/Maze-Solver/src/app/Version.java File Reference . . . 118
10.5 C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentCli.java File Reference 118
10.6 C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentResult.java File Reference . . . 119
10.7 C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java File Reference . . . 119
10.8 C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java File Reference . . . 120
10.9 C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuiteIO.java File Reference . . . 121
10.10 C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java File Reference . . . 121
10.11 C:/Users/ramse/Github/Maze-Solver/src/maze/map/MazeFileIO.java File Reference . . . 122
10.12 C:/Users/ramse/Github/Maze-Solver/src/maze/search/Astar.java File Reference . . . 122
10.13 C:/Users/ramse/Github/Maze-Solver/src/maze/search/GreedySearch.java File Reference 123

10.14 C:/Users/ramse/Github/Maze-Solver/src/maze/search/MazeSolver.java File Reference . 123
10.15 C:/Users/ramse/Github/Maze-Solver/src/maze/search/SearchStrategy.java File Reference 123
10.16 C:/Users/ramse/Github/Maze-Solver/src/maze/search/UCS.java File Reference 124
10.17 C:/Users/ramse/Github/Maze-Solver/src/maze/search/UninformedSearch.java File Reference 124
10.18 C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultsPanel.java File Reference 125
10.19 C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultTableModel.java File Reference 125
10.20 C:/Users/ramse/Github/Maze-Solver/src/ui/MazeRenderer.java File Reference 126
10.21 C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfigurationTable.java File Reference 127
10.22 C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfigurationTableModel.java File Reference 127
10.23 C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteConfigurationPanel.java File Reference 128
10.24 C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteWindow.java File Reference 128
10.25 C:/Users/ramse/Github/Maze-Solver/src/ui/UITheme.java File Reference 129

Chapter 1

Maze-Solver 0.8.1

Maze-Solver 0.8.1 is a Java Swing desktop app for loading, visualizing, and solving grid mazes using DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗. Maze files support optional per-direction movement costs while preserving the original unweighted format.

It is designed for both interactive exploration and large-map performance testing, with a paint-based renderer that supports very large mazes efficiently.

1.1 Table of Contents

1. Overview
2. Features
3. Requirements
4. Quick Start
5. How to Use
6. Maze File Format
7. Included Maze Files
8. Algorithms
9. Rendering and Performance
10. Project Structure
11. API and Extension Notes
12. Launch From Release JAR
13. Troubleshooting
14. Development Notes
15. License

1.2 Overview

Maze-Solver has six main responsibilities:

1. Load human-readable .msmap maze files from disk.
2. Validate and visualize the maze graph.
3. Run DFS, BFS, UCS, Greedy Best-First Search, or weighted A∗ incrementally or instantly.
4. Configure reproducible multi-algorithm experiment suites and export results.
5. Review exact experiment maps in the animated renderer.
6. Show traversal progress and final path with clear visual semantics.

1.3 Features

- Title-screen startup flow with import-or-generate selection.
- Random maze generation using traditional corridor carving.
- Interactive solver controls for start, pause, restart, algorithm, and delay.
- Instant solve mode that runs to completion in a background thread.
- In-session map controls for loading a different map, saving the current map, and generating a new random map.
- Paint-based grid rendering for all maze sizes.
- Viewport clipping so only visible cells are painted.
- Zoom controls (+/- and Ctrl + mouse wheel).
- Gridline overlay for easier node-by-node visibility.
- Start/goal color preservation after final path draw.
- Debug diagnostics for maze import and neighbor consistency checks.
- Optional weighted movement data using a compact [North, East, South, West] format.
- Optional grid overlay for viewing directional movement costs. Equal opposite costs share one edge label; asymmetric costs show separate directional labels.
- Default startup solve settings: A∗ algorithm and 200 ms step delay.
- Application Settings can configure the maximum JVM heap for the next launch. Java cannot increase the heap limit of an already-running process; the launcher reads the saved setting and starts Java with the corresponding -Xmx option.
- Configurable test-suite workflow from the title screen.
- Seeded experiment generation with adjustable map size, interior obstacle density, reachability, endpoint placement, and movement-cost policies.
- Generated experiment maps use a seeded traditional corridor-maze backbone. The configured blocked percentage is applied by opening additional cells or removing dead-end corridors, so density changes preserve long orthogonal walls and one-cell-wide passages instead of producing speckled noise.
- Guaranteed-solvable generation retries disconnected layouts and uses a seeded winding connector only as a final fallback. Fixed endpoints are preserved during density adjustment.
- Shared-map comparisons across any selected subset of the five algorithms.
- Suite save/load, progress and cancellation, per-result animated review, suite map export, and UTF-8 CSV export.

1.4 Requirements

- Java JDK 21 (LTS) or newer, including the jpackage tool for executable generation.
- Windows, macOS, or Linux with desktop GUI support.
- No external dependencies required.
- Release version: 0.8.1 (also exposed by app.Version.RELEASE).

1.5 Quick Start

From the repository root:

1. Compile (PowerShell)

```
New-Item -ItemType Directory -Force build\classes | Out-Null
$sources = @((Get-ChildItem -Filter *.java src -Recurse).FullName) +
    @((Get-ChildItem -Filter *.java test\java -Recurse).FullName)
javac -Xlint:all -d build\classes $sources
```

1. Run

```
java -cp build\classes app.MS
```

Run the dependency-free regression suite with:

```
java -cp build\classes test.RegressionTests
```

Run an experiment suite without opening Swing:

```
java -cp build\classes experiments.ExperimentCli `
    --suite test\resources\suites\test_testSuiteExample.mstests `
    --mode SEQUENTIAL `
    --monitor-ms 1000 `
    --csv build\results\cli-results.csv
```

The command-line runner prints periodic progress, heap usage, active worker counts, and interval worker peaks. Use FIXED_PARALLEL or ADAPTIVE_PARALLEL to benchmark the execution modes, and --repeat N to repeat the same seeded suite. Press Ctrl+C to request cooperative cancellation. Use --quiet when only the final timing summary is needed. This runner uses the same ExperimentRunner as the application, so command line runs exercise the production scheduler without UI rendering overhead.

Startup sequence:

1. Title screen appears.
2. Choose Import Maze or Generate Random.
3. If importing, select a maze text file. If generating, enter rows and columns, then optionally enable movement weights with a range from 1 through 100.
4. Solve interactively.

Use Back to Main Menu in the maze controls to leave an interactive session without exiting the application. The test-suite editor has the same navigation button, and closing either secondary window returns to the title screen. The title screen is hidden while a secondary screen is open so each menu is only shown once.

To configure experiments, choose Configure Tests on the title screen. Add one row per test condition, edit the row values, select algorithms and iterations, then choose Run Tests. Each generated-map iteration creates one map and uses that same map for every selected algorithm. Generated layouts retain the suite's row/column, blocked-percent, reachability, endpoint, seed, and movement-cost settings. Saved-map rows reuse their embedded map for every iteration. Automatic seeds are materialized when a suite is saved so later generated-map runs can be reproduced. Results can be exported manually or automatically to CSV, and unique maps can be exported for later inspection.

The test editor uses dropdowns for reachability, cost, endpoint, and map-source preferences. The Algorithms cell provides a checkbox multi-select, and boolean options use checkboxes. Numeric values are normalized to integers when editing ends. In RANDOM and DISTANCE endpoint modes, start and goal coordinates are disabled; DISTANCE enables the target-distance field.

Each row can use an automatic seed, a manually entered seed, or a saved map. Saved maps are imported through the row editor and embedded in the suite file using UTF-8-safe encoding, so the suite does not depend on the original map file remaining available. Saved-map results identify the source as SAVED↩ _MAP and display N/A (saved map) instead of a generated seed. Test suites use the human-readable .mstests extension; individual maps use the human-readable .msmap extension. Import and save dialogs filter and normalize to their corresponding extension and open in the application directory. A saved suite contains applicationVersion=0.8.1 and version=1; the former identifies the application release and the latter is the stable suite schema version. Older schema-version-1 suites remain readable.

Before measured runs begin, the experiment runner performs a short JVM warm-up using each selected algorithm. Warm-up solves are excluded from the results, progress count, and timing metrics so class loading and JIT compilation are less likely to skew the first measured iteration.

Experiment suites run sequentially by default. Select Fixed parallel or Adaptive parallel in the test-suite editor to run independent algorithms for each generated map concurrently. Sequential mode remains available for compatibility checks and comparisons. Each generated map is shared by the selected algorithms for that iteration, and results remain ordered by test condition, iteration, and algorithm.

Parallel execution is most useful for larger maps or suites with several selected algorithms. Small suites may show little improvement because there is not enough work to keep all workers busy. Select Sequential to retain the original execution path for comparison and compatibility testing. Parallel execution improves throughput primarily for larger maps; small maps may show little memory or elapsed-time difference because they do not create enough work to saturate the workers.

Experiment results retain only measurements, configuration metadata, and the effective map seed. Generated maps are recreated from that seed only when a result is reviewed, which avoids retaining large serialized maps for every algorithm run. The results table is paged with selectable page sizes of 10, 25, 50, or 100 rows. The footer shows live JVM memory utilization, supports direct page-number entry, and supports jumps of 1, 10, or 100 pages in either direction. The progress status identifies the most recently completed test condition, and heap exhaustion reports the active condition so large suites can be split into smaller batches. Enable Follow latest results in the results footer to keep the table on the dynamically growing final page while a suite runs. Using any paging control pauses following so earlier result pages can be inspected; enable it again to resume following the newest page. The Results view selector can switch between the paged table and a live solve log. The live log appends each completed solve with its condition, iteration, algorithm, solved status, path cost, explored-node count, and runtime, and automatically follows the newest entry. It is intentionally bounded to the most recent 500 solve entries; the complete results remain available in the table and exported result data. The live log is split into a wide solver-activity pane and a narrower map-generation pane. The map pane records map readiness, solver dispatch, and map completion events, along with the number of generated maps currently loaded in memory waiting for solver work. That loaded-map count is distinct from maps still being generated and is also shown on the map progress bar. During a run, Pause cooperatively stops new map generation and solver work at task boundaries; Resume continues the same run. Cancel stops the

run and clears retained maps, scheduler state, and completion queues so large generated maps can be reclaimed promptly. While solves are active, the log places live worker lines above the completed history and updates their nodes explored, current path cost, and elapsed time in place. Progress notifications are throttled so the diagnostic view does not receive one UI event per visited node. A future small-suite race view may make this comparison more visual.

Seed behavior is stable for the complete execution of each test case. Manual seed mode reuses the configured seed for every iteration, while automatic seed mode generates one seed once and reuses it for every iteration. Consequently, generated iterations in the same row recreate the same map; repeated solver runs are directly comparable rather than independent random samples. Saved-map mode reuses the embedded map and does not use a generated seed. Every selected algorithm in an iteration receives the same read-only map.

Approximate-distance endpoint selection first uses seeded random sampling for speed. If sampling misses a valid pair, the generator performs a deterministic distance-offset search before rejecting the layout. This keeps results reproducible while avoiding failures caused only by random endpoint sampling.

The test-suite results tab includes a Review button for each result. It reimports the exact map snapshot used by that result and opens the animated MazeRenderer with the result's algorithm selected. Export Maps creates a <suite-name>-maps folder under the selected directory and writes one complete saveable map file for each unique map. Repeated iterations are grouped in the filename, such as iterations-1-3, while distinct maps retain their iteration numbers. Seed-generated maps are converted into the same format used by imported maps, making them portable for later inspection or reuse.

1.6 How to Use

After loading or generating a maze:

- Start: begin or resume timer-based solving.
- Pause: stop solving without clearing current search state.
- Restart: clear colors and reset the active algorithm state.
- Load Map: replace the current session maze with another file.
- Save Map: export the currently loaded maze to a text file.
- Generate Map: create and load a new random maze without leaving the solver window.
- Algorithm: choose DFS, BFS, Uniform Cost Search, Greedy Best-First Search, or weighted A$*$.
- Step Delay (ms): set timer delay for step-by-step mode (1..5000).
- Instant: solve to completion immediately (async worker).
- Show Weights: overlay movement costs on grid edges. Labels appear when zoom is at least 20 pixels per cell and use a slightly larger font for readability. Equal opposite-direction costs use one number; different costs use direction markers (>, <, v, ^) and two numbers. During solving, labels next to visited nodes use a subdued gray style. Once solved, only the exact consecutive edges in the final path use an orange-on-gold highlight. Solver updates repaint only the changed cells and adjacent edge-label area; the entire grid is not redrawn for each visited node. The localized repaint includes a small pixel margin because labels are centered on cell boundaries.
- Current Cost: the cost from the start to the most recently visited node. This tracks only that node's selected parent route, not all explored branches.
- Final Cost: the total movement cost of the returned route, or -- when no solution has been found.

- Solve Time: elapsed time spent inside solver search-step calls. Timer delays, UI repainting, and animation intervals are outside those measurements, so animated and instant solving times are directly comparable. The console also prints a diagnostic line containing the result, interval, algorithm, visited node count, final path cost, compute time, and wall-clock session time after each completed solve.
- Zoom +/-: adjust cell size.
- Ctrl + Mouse Wheel: zoom while cursor is over the maze viewport.

Saved maze files automatically receive a .msmap extension when one is not provided. The import dialog filters for .msmap maze files; reopening the dialog is sufficient to see a newly saved file, and restarting the application is not required.

Random generation details:

- Uses a depth-first backtracking corridor-carving strategy for a classic maze look.
- Keeps the outer border blocked.
- Ensures the configured goal cell is connected to the carved maze.
- Extends carving into final interior row/column on even dimensions to avoid persistent right/bottom wall strips.
- The generator can optionally enable movement weights and accepts an inclusive minimum and maximum range from 1 through 100. These limits are centralized in Maze.MIN_MOVEMENT_↩ COST and Maze.MAX_MOVEMENT_COST.
- The default maximum in the generator dialog is 10.
- The generator defaults to using the same positive integer cost in both directions of each connection. Clear the same-cost checkbox to generate independent integer costs for reverse directions.

1.7 Maze File Format

Expected format:

1. Row count
2. Column count
3. start: row,col
4. end: row,col
5. One row per maze row using:
   - 1 for traversable/open
   - 0 for blocked/wall
6. Optional WEIGHTS section with one four-value row per grid coordinate.

Example:

```
4
5
start: 3,0
end: 0,4
11111
10001
10101
11111
```

Legacy files without a WEIGHTS section remain valid. Every adjacent open-cell connection receives a default movement cost of 1.

1.7.1 Weighted map format

The optional section follows the complete text layout:

```
4
5
start: 3,0
end: 0,4
11111
10001
10101
11111

WEIGHTS
0,1,1,0
0,1,0,1
0,1,0,1
0,1,0,1
0,0,1,1
1,0,1,0
0,0,0,0
0,0,0,0
0,0,0,0
1,0,1,0
1,1,0,0
0,1,0,1
0,1,0,1
0,1,0,1
1,0,0,1
1,1,0,0
1,1,0,1
1,1,0,1
1,1,0,1
1,0,0,1
0,0,0,0
0,0,0,0
```

There is one weight row for every coordinate in row-major order, including blocked cells. Each row is ordered [North, East, South, West] and uses:

- 0: no permitted movement in that direction.
- 1 or greater: the positive cost of moving in that direction.

For an open cell, a direction leading outside the maze or into a blocked cell must have cost 0. Blocked cells must contain four zero values. Costs may be asymmetric, so the cost from A to B does not have to equal the cost from B to A. The current map model preserves geometric neighbor references separately from movement availability: a neighboring node is traversable only when its movement cost is greater than zero.

MazeSolver uses movement costs when determining whether an edge is traversable and when calculating weighted search scores. UCS and A$*$ guarantee a minimum-cost route when a route exists because all enabled movement costs are positive.

Validation rules enforced by import:

- Dimensions must be positive.
- Layout row count must match declared row count.
- Each layout row width must match declared column count.
- Only 0 and 1 are valid cell values.
- Start and end must be in bounds and on traversable cells.
- If WEIGHTS is present, it must contain exactly rows $*$ columns rows.
- Each weight row must contain exactly four integer values. Enabled movement costs must be in the range 1..100; 0 is reserved for disabled or non-existent directions.
- Weight rows must match the layout topology.

1.8 Included Maze Files

Located in src:

- ∗.msmap: human-readable individual maze map files.
- test_100x100.msmap: medium map for standard performance checks.
- test_346x346.msmap: larger map for stress testing.
- test_1000x1000.msmap: very large map for renderer scalability tests.
- test_weighted_100x100.msmap: weighted 100x100 example with directional costs.

1.9 Algorithms

The solver supports:

- DFS: depth-first exploration, memory-light but not shortest-path or minimum-cost guaranteed.
- BFS: breadth-first exploration, shortest by number of moves when all enabled moves have equal cost; it respects disabled edges but does not optimize weighted path cost.
- Uniform Cost Search: uninformed priority search ordered by accumulated route cost g(n). With positive movement costs, it returns a minimum-cost path.
- Greedy Best-First Search: priority search ordered only by the Manhattan heuristic h(n). It can explore fewer nodes, but it is not guaranteed to return a minimum-cost path.
- A∗: weighted informed search using cumulative directional movement costs and the Manhattan-distance heuristic. Since valid movement costs are at least 1, the heuristic remains an admissible lower bound and A∗ returns a minimum-cost path when one exists.

All algorithms use the [North, East, South, West] movement costs from the loaded maze. A neighboring cell with cost 0 is treated as an unavailable edge. A∗ stores cumulative costs as long values to avoid narrowing the sum of many individual movement costs.

A∗ heuristic used:

$$h(n) = |r_n - r_g| + |c_n - c_g|$$

where r_n,c_n is current node and r_g,c_g is goal.

The priority functions used by the informed searches are:

- UCS: f(n) = g(n)
- Greedy Best-First Search: f(n) = h(n)
- A∗: f(n) = g(n) + h(n)

The solver uses graph-search repeated-state tracking for all priority-based algorithms. UCS and A∗ ignore stale frontier entries after a cheaper route is discovered. Greedy keeps the first discovered route to each cell, consistent with its heuristic-only ordering and non-optimality guarantee.

The Manhattan heuristic is admissible because every enabled move costs at least 1. It is also consistent for this four-directional maze: moving to an adjacent cell changes Manhattan distance by at most 1, while the transition cost is at least 1.

1.10 Rendering and Performance

Current renderer behavior:

- One paint-based path for every maze size.
- Cell colors are cached in a backing array.
- Only visible clip-region cells are painted each frame.
- Gridlines are rendered with a subtle alpha stroke.
- Start and goal labels (S/G) are drawn at larger zoom levels.

Color semantics:

- Start: green
- Goal: blue
- Open: white
- Blocked: dark gray
- Visited: configurable (default yellow)
- Final path: cyan

When the weight overlay is enabled, 0 means movement is disabled and positive numbers are movement costs. The overlay preserves directional asymmetry: for a horizontal connection, > is movement to the east and < is movement to the west; for a vertical connection, v is movement to the south and ^ is movement to the north.

After a solved run, start and goal are restored to semantic colors.

1.11 Project Structure

```
Maze-Solver/
  README.md
  TODO.md
  LICENSE
  src/
    app/                  application entry point
    maze/
      map/                maze model, generation, and map persistence
      search/             solver facade and search strategies
    experiments/          experiment configuration and execution
    ui/                   Swing renderer and test-suite window
  test/
    java/                 regression tests
    resources/
      maps/               .msmap fixtures
      suites/             .mstests fixtures
  doc/                    generated Doxygen HTML documentation
```

Core packages and classes:

- app.MS: application entry point for release 0.8.1.
- app.Version: authoritative application release metadata.

- maze.map.Maze: model for occupancy, neighbors, directional costs, start, goal, generation, and validation.
- maze.map.MazeFileIO: dedicated .msmap parser and serializer used by the compatibility import/export methods on maze.map.Maze.
- maze.search.MazeSolver: public incremental solver facade, lifecycle, metrics, and path reconstruction.
- maze.search.SearchStrategy, maze.search.UninformedSearch, maze.search.UCS, maze.search.GreedySearch, and maze.search.Astar: internal incremental algorithm strategies used by the solver facade. UninformedSearch handles both DFS and BFS through its explicit TraversalMode frontier policy.
- ui.MazeRenderer: UI flow, controls, rendering, and solve orchestration.
- experiments.TestSuite: validated experiment-row and suite configuration.
- experiments.ExperimentResult: immutable per-algorithm experiment metrics.
- experiments.ExperimentRunner: non-UI seeded experiment execution with progress and cancellation.
- experiments.TestSuiteIO: versioned .mstests persistence and CSV export.
- ui.TestSuiteWindow: Swing configuration, execution, results, map review, and export workflow and coordination.
- ui.TestConfigurationTableModel and ui.ExperimentResultTableModel: extracted configuration/result table state.
- ui.TestConfigurationTable: context-sensitive saved-map and endpoint editability behavior.
- ui.TestSuiteTableComponents: package-private reusable experiment-table editors and renderers, including algorithm selection, saved-map import, and Review actions. These components are wired by the Swing table classes.
- ui.TestSuiteConfigurationPanel and ui.ExperimentResultsPanel: extracted Swing layout and action wiring for configuration and result areas.
- test.RegressionTests: dependency-free checks for weighted maps, all algorithms, reproducibility, persistence, CSV export, and cancellation.

Test fixtures are kept separate from application sources:

- test/resources/maps/: sample .msmap files used by regression checks.
- test/resources/suites/: sample .mstests experiment suite.

1.12 API and Extension Notes

Useful extension points:

```
Maze.Node start = maze.getStart();
MazeRenderer.setNodeColor(start, java.awt.Color.MAGENTA);
MazeRenderer.resetNodeColor(start);
```

Typical programmatic solve usage:

```
Maze maze = Maze.importFromFile(
        "test/resources/maps/test_346x346.msmap");
Maze.Node next = maze.getStart().getNeighbor(Maze.Dir.EAST);
if (next != null && maze.getMoveCost(maze.getStart(), next) > 0) {
    System.out.println("East movement is enabled.");
```

```
}
MazeSolver solver = new MazeSolver();
solver.setMaze(maze);
solver.setAlg("A*");

while (!solver.isFinished()) {
    solver.updateSolve();
}

if (solver.isSolved()) {
    java.util.List<java.awt.Point> path = solver.getFinalPath();
    System.out.println("Path nodes: " + path.size());
    System.out.println("Path cost: " + solver.getPathCost());
}
```

1.13 Launch From Release JAR

If you downloaded a release artifact from GitHub (or received a provided prebuilt JAR), launch it as follows:

1. Install Java JDK 21 (LTS) or newer.
2. Download the JAR and extract it first if it is packaged in a zip.
3. Open a terminal in the folder containing the JAR.
4. Run:

```
java -jar MazeSolver.jar
```

If your file has a different name, replace MazeSolver.jar with the actual filename.

For builds distributed with external assets, keep this layout:

```
release-folder/
  MazeSolver.jar
  images/
    TitleImage.jpg
```

If the app starts but the title image does not appear, ensure the images folder is present next to the JAR. The title screen images can be loaded from images/TitleImage.jpg.

1.14 Troubleshooting

1. App does not start

- Confirm javac and java are on PATH.
- Rebuild by compiling application sources in src\ and regression tests in test\java\ from the repository root.

1. Maze fails to load

- Check file format and dimensions.
- Ensure start/end are on traversable cells (1).

1. Slow behavior on huge maps

- Use lower zoom to reduce overdraw.
- Prefer Instant mode for completion-only runs.

1. Unexpected debug output volume

- The renderer prints diagnostics during load.
- This is useful for validation but verbose on large maps.

1.15 Development Notes

Useful commands from repo root:

```
New-Item -ItemType Directory -Force build\classes | Out-Null
$sources = @(
    (Get-ChildItem -Filter *.java src -Recurse).FullName
    (Get-ChildItem -Filter *.java test\java -Recurse).FullName
)
javac -Xlint:all -d build\classes $sources
java -cp build\classes app.MS
```

Convenience scripts are available under scripts\:

```
.\scripts\compile-and-run.ps1
.\scripts\build-executable.ps1
```

The first script compiles the application and regression tests, runs the regression suite, and launches the Swing application. The second creates a self-extracting single-file Windows executable: dist\MazeSolver-0.8.1.exe. The executable embeds the complete application image and Java runtime, extracts to a temporary directory when launched, waits for the application to close, and removes its temporary files. It can be distributed as one file and does not require Java on the target machine. Normal launches do not open a terminal window. For troubleshooting, launch the executable with the optional debug flag:

```
.\dist\MazeSolver-0.8.1.exe --debug
```

1.15.1 Developer Notes: experiment execution

ExperimentRunner keeps the UI-independent execution path separate from Swing and preserves deterministic result ordering. Sequential mode executes one map and its algorithms directly. Fixed parallel mode uses the processor-aware worker cap; adaptive mode adjusts that cap from CPU load and heap headroom.

Parallel execution uses one priority executor. Map-generation tasks are prioritized while the memory frontier is open, but ready maps and queued solver tasks are bounded to approximately two worker-widths. This bound is important: queued solver tasks capture their shared map, so an unbounded queue would retain large maps even after generation has finished.

The coordinator is completion-driven. Worker completion releases a notification permit, avoiding fixed polling delays. A map is removed from scheduler state after its final selected algorithm completes; the resulting ExperimentResult contains measurements and metadata, not the map object. Clearing scheduler collections during cancellation and shutdown makes those maps eligible for garbage collection promptly. The JVM may retain committed heap for reuse, so process working-set size is not the same as live object usage.

Admission reservations estimate map cells, solver tables, and transient solver overhead with a safety factor. They are a scheduling guard, not a heap measurement. Diagnostics therefore expose both reservation state and JVM heap state. On hybrid CPUs, Runtime.availableProcessors() reflects processors exposed to the JVM; process affinity can be used for controlled comparisons.

Debug mode opens a terminal window while the application runs. Building it requires JDK 21 or newer, jpackage, and the .NET 8 SDK for the native self-extractor wrapper.

On Linux, make the equivalent scripts executable once and run:

```
chmod +x scripts/compile-and-run.sh scripts/build-executable.sh
./scripts/compile-and-run.sh
./scripts/build-executable.sh
```

The Linux application image is created at dist/MazeSolver/bin/MazeSolver, and the portable bundle is dist/MazeSolver-portable-linux.zip. These bundles include the Java runtime and do not require Java to be installed on the target machine. The generated build/ and dist/ directories are ignored by Git.

Optional cleanup of compiled artifacts:

```
Remove-Item -Recurse -Force build
```

Validate JavaDoc without generating the Doxygen/PDF artifacts:

```
$files = (Get-ChildItem src -Filter *.java -Recurse).FullName
javadoc -quiet -Xdoclint:all -d "$env:TEMP\maze-javadoc" $files
```

Doxygen is configured by doc/Doxyfile. Generate the HTML reference and release PDF with the documentation script:

```
.\scripts\generate-documentation.ps1
```

The script checks for Doxygen and XeLaTeX. If either is missing, it asks before installing it through an available package manager. It generates the HTML reference, builds the PDF, and removes only the generated doc\latex folder. The remaining documentation outputs are doc\html, doc\Maze-Solver-docs↩ _V0.8.1.pdf, and doc\Doxyfile.

On Linux, use the equivalent script:

```
chmod +x scripts/generate-documentation.sh
./scripts/generate-documentation.sh
```

1.16 License

This project is licensed under the terms in LICENSE.

Chapter 2

Directory Hierarchy

2.1 Directories

app . . . 25
ApplicationPreferences.java . . . 117
MS.java . . . 117
Version.java . . . 118
experiments . . . 25
ExperimentCli.java . . . 118
ExperimentResult.java . . . 119
ExperimentRunner.java . . . 119
TestSuite.java . . . 120
TestSuiteIO.java . . . 121
map . . . 25
Maze.java . . . 121
MazeFileIO.java . . . 122
maze . . . 25
map . . . 25
Maze.java . . . 121
MazeFileIO.java . . . 122
search . . . 26
Astar.java . . . 122
GreedySearch.java . . . 123
MazeSolver.java . . . 123
SearchStrategy.java . . . 123
UCS.java . . . 124
UninformedSearch.java . . . 124
search . . . 26
Astar.java . . . 122
GreedySearch.java . . . 123
MazeSolver.java . . . 123
SearchStrategy.java . . . 123
UCS.java . . . 124
UninformedSearch.java . . . 124
src . . . 26
app . . . 25
ApplicationPreferences.java . . . 117
MS.java . . . 117
Version.java . . . 118

experiments . . . 25
ExperimentCli.java . . . 118
ExperimentResult.java . . . 119
ExperimentRunner.java . . . 119
TestSuite.java . . . 120
TestSuiteIO.java . . . 121
maze . . . 25
map . . . 25
Maze.java . . . 121
MazeFileIO.java . . . 122
search . . . 26
Astar.java . . . 122
GreedySearch.java . . . 123
MazeSolver.java . . . 123
SearchStrategy.java . . . 123
UCS.java . . . 124
UninformedSearch.java . . . 124
ui . . . 26
ExperimentResultsPanel.java . . . 125
ExperimentResultTableModel.java . . . 125
MazeRenderer.java . . . 126
TestConfigurationTable.java . . . 127
TestConfigurationTableModel.java . . . 127
TestSuiteConfigurationPanel.java . . . 128
TestSuiteWindow.java . . . 128
UITheme.java . . . 129
ui . . . 26
ExperimentResultsPanel.java . . . 125
ExperimentResultTableModel.java . . . 125
MazeRenderer.java . . . 126
TestConfigurationTable.java . . . 127
TestConfigurationTableModel.java . . . 127
TestSuiteConfigurationPanel.java . . . 128
TestSuiteWindow.java . . . 128
UITheme.java . . . 129

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

app 27
experiments 27
maze.map 27
maze.search 28
ui 28

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

maze.search.MazeSolver.Algorithm . . . 29
app.ApplicationPreferences . . . 30
experiments.ExperimentRunner.CancellationToken . . . 32
    experiments.ExperimentRunner.ExecutionControl . . . 36
experiments.TestSuite.CostMode . . . 32
maze.map.Maze.Dir . . . 33
experiments.TestSuite.EndpointMode . . . 35
experiments.TestSuite.ExecutionMode . . . 37
experiments.ExperimentCli . . . 38
experiments.ExperimentResult . . . 39
experiments.ExperimentRunner . . . 48
maze.map.Maze.GeneratedExperimentMap . . . 54
experiments.ExperimentRunner.MapProgressListener . . . 57
experiments.TestSuite.MapSource . . . 58
maze.map.Maze . . . 59
maze.map.MazeFileIO . . . 71
ui.MazeRenderer . . . 73
maze.search.MazeSolver . . . 76
app.MS . . . 81
maze.map.Maze.NeighborValidationResult . . . 82
maze.map.Maze.Node . . . 83
experiments.ExperimentRunner.ProgressListener . . . 87
experiments.TestSuite.ReachabilityMode . . . 87
maze.search.SearchStrategy . . . 88
experiments.ExperimentRunner.SolveProgress . . . 89
experiments.ExperimentRunner.SolveProgressListener . . . 91
experiments.ExperimentRunner.TaskPriority . . . 92
experiments.TestSuite.TestCase . . . 93
experiments.TestSuite . . . 106
experiments.TestSuiteIO . . . 111
ui.TestSuiteWindow . . . 114
maze.search.UninformedSearch.TraversalMode . . . 115
app.Version . . . 115
ui.MazeRenderer.WeightDisplayState . . . 116

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

maze.search.MazeSolver.Algorithm
: Search strategies supported by this solver . . . 29

app.ApplicationPreferences
: Stores user-level application preferences that are needed before the JVM starts, such as the requested maximum heap size . . . 30

experiments.ExperimentRunner.CancellationToken
: Allows a caller to stop a long-running experiment . . . 32

experiments.TestSuite.CostMode
: Movement-cost distribution used by an experiment row . . . 32

maze.map.Maze.Dir
: Cardinal directions used for indexing node neighbors . . . 33

experiments.TestSuite.EndpointMode
: How an experiment row chooses its start and goal coordinates . . . 35

experiments.ExperimentRunner.ExecutionControl
: Adds cooperative pausing to cancellation for interactive runs . . . 36

experiments.TestSuite.ExecutionMode
: Execution strategy for experiment solver runs . . . 37

experiments.ExperimentCli
: Headless command-line entry point for running and benchmarking experiment suites without starting Swing . . . 38

experiments.ExperimentResult
: Immutable measurements from one algorithm run on one generated map . . . 39

experiments.ExperimentRunner
: Executes configured experiment rows without Swing or rendering overhead . . . 48

maze.map.Maze.GeneratedExperimentMap
: Metadata for one generated experiment map . . . 54

experiments.ExperimentRunner.MapProgressListener
: Receives progress after each map has finished generating . . . 57

experiments.TestSuite.MapSource
: Source used to obtain the map for an experiment row . . . 58

maze.map.Maze
: Core maze model storing occupancy, neighbor links, and start/goal nodes . . . 59

maze.map.MazeFileIO
: Reads and writes the human-readable .msmap format . . . 71

ui.MazeRenderer
: Renderer and UI controller for the maze application . . . 73

maze.search.MazeSolver
Incremental maze solver that supports DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗ . . . 76
app.MS
Demo application entry point . . . 81
maze.map.Maze.NeighborValidationResult
Validation result for neighbor consistency checks . . . 82
maze.map.Maze.Node
Represents a single cell in the maze grid . . . 83
experiments.ExperimentRunner.ProgressListener
Receives progress after each completed algorithm run . . . 87
experiments.TestSuite.ReachabilityMode
Whether generated maps must contain a route between their endpoints . . . 87
maze.search.SearchStrategy
Internal incremental search strategy used by MazeSolver . . . 88
experiments.ExperimentRunner.SolveProgress
Immutable diagnostic snapshot for one active solver worker . . . 89
experiments.ExperimentRunner.SolveProgressListener
Receives throttled progress from solver workers while they are active . . . 91
experiments.ExperimentRunner.TaskPriority . . . 92
experiments.TestSuite.TestCase
Describes one row in a test suite . . . 93
experiments.TestSuite
Configuration model for repeatable maze-solving experiments . . . 106
experiments.TestSuiteIO
Persistence and reporting utilities for experiment suites and results . . . 111
ui.TestSuiteWindow
Swing editor and execution window for configurable experiment suites . . . 114
maze.search.UninformedSearch.TraversalMode . . . 115
app.Version
Release metadata for the Maze-Solver application . . . 115
ui.MazeRenderer.WeightDisplayState
Describes how a weight label should be styled based on the solve state . . . 116

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

C:/Users/ramse/Github/Maze-Solver/src/app/ApplicationPreferences.java . . . 117
C:/Users/ramse/Github/Maze-Solver/src/app/MS.java . . . 117
C:/Users/ramse/Github/Maze-Solver/src/app/Version.java . . . 118
C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentCli.java . . . 118
C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentResult.java . . . 119
C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java . . . 119
C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java . . . 120
C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuiteIO.java . . . 121
C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java . . . 121
C:/Users/ramse/Github/Maze-Solver/src/maze/map/MazeFileIO.java . . . 122
C:/Users/ramse/Github/Maze-Solver/src/maze/search/Astar.java . . . 122
C:/Users/ramse/Github/Maze-Solver/src/maze/search/GreedySearch.java . . . 123
C:/Users/ramse/Github/Maze-Solver/src/maze/search/MazeSolver.java . . . 123
C:/Users/ramse/Github/Maze-Solver/src/maze/search/SearchStrategy.java . . . 123
C:/Users/ramse/Github/Maze-Solver/src/maze/search/UCS.java . . . 124
C:/Users/ramse/Github/Maze-Solver/src/maze/search/UninformedSearch.java . . . 124
C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultsPanel.java . . . 125
C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultTableModel.java . . . 125
C:/Users/ramse/Github/Maze-Solver/src/ui/MazeRenderer.java . . . 126
C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfigurationTable.java . . . 127
C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfigurationTableModel.java . . . 127
C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteConfigurationPanel.java . . . 128
C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteWindow.java . . . 128
C:/Users/ramse/Github/Maze-Solver/src/ui/UITheme.java . . . 129

Chapter 7

Directory Documentation

7.1 C:/Users/ramse/Github/Maze-Solver/src/app Directory Reference

Files

- file ApplicationPreferences.java
- file MS.java
- file Version.java

7.2 C:/Users/ramse/Github/Maze-Solver/src/experiments Directory Reference

Files

- file ExperimentCli.java
- file ExperimentResult.java
- file ExperimentRunner.java
- file TestSuite.java
- file TestSuiteIO.java

7.3 C:/Users/ramse/Github/Maze-Solver/src/maze/map Directory Reference

Files

- file Maze.java
- file MazeFileIO.java

7.4 C:/Users/ramse/Github/Maze-Solver/src/maze Directory Reference

Directories

- directory map
- directory search

7.5 C:/Users/ramse/Github/Maze-Solver/src/maze/search Directory Reference

Files

- file Astar.java
- file GreedySearch.java
- file MazeSolver.java
- file SearchStrategy.java
- file UCS.java
- file UninformedSearch.java

7.6 C:/Users/ramse/Github/Maze-Solver/src Directory Reference

Directories

- directory app
- directory experiments
- directory maze
- directory ui

7.7 C:/Users/ramse/Github/Maze-Solver/src/ui Directory Reference

Files

- file ExperimentResultsPanel.java
- file ExperimentResultTableModel.java
- file MazeRenderer.java
- file TestConfigurationTable.java
- file TestConfigurationTableModel.java
- file TestSuiteConfigurationPanel.java
- file TestSuiteWindow.java
- file UITheme.java

Chapter 8

Namespace Documentation

8.1 Package app

Classes

- class ApplicationPreferences

  Stores user-level application preferences that are needed before the JVM starts, such as the requested maximum heap size.
- class MS

  Demo application entry point.
- class Version

  Release metadata for the Maze-Solver application.

8.2 Package experiments

Classes

- class ExperimentCli

  Headless command-line entry point for running and benchmarking experiment suites without starting Swing.
- class ExperimentResult

  Immutable measurements from one algorithm run on one generated map.
- class ExperimentRunner

  Executes configured experiment rows without Swing or rendering overhead.
- class TestSuite

  Configuration model for repeatable maze-solving experiments.
- class TestSuiteIO

  Persistence and reporting utilities for experiment suites and results.

8.3 Package maze.map

Classes

- class Maze

  Core maze model storing occupancy, neighbor links, and start/goal nodes.
- class MazeFileIO

  Reads and writes the human-readable .msmap format.

8.4 Package maze.search

Classes

- class Astar

  Incremental weighted A∗ strategy using Manhattan distance as its heuristic.
- class GreedySearch

  Greedy Best-First Search strategy using Manhattan distance as its priority.
- class MazeSolver

  Incremental maze solver that supports DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗.
- interface SearchStrategy

  Internal incremental search strategy used by MazeSolver.
- class UCS

  Uniform-Cost Search strategy.
- class UninformedSearch

  Incremental implementation for the uninformed DFS and BFS algorithms.

8.5 Package ui

Classes

- class ExperimentResultsPanel

  Layout and actions for experiment execution controls and results.
- class ExperimentResultTableModel

  Read-only table model for measured experiment results.
- class MazeRenderer

  Renderer and UI controller for the maze application.
- class TestConfigurationTable

  Configuration table with context-sensitive editability and disabled-cell rendering for saved-map and endpoint modes.
- class TestConfigurationTableModel

  Table model for editable experiment configurations.
- class TestSuiteConfigurationPanel

  Layout and actions for editing the suite name and test configurations.
- class TestSuiteWindow

  Swing editor and execution window for configurable experiment suites.
- class UITheme

  Shared visual language for the Swing screens.

Chapter 9

Class Documentation

9.1 maze.search.MazeSolver.Algorithm Enum Reference

Search strategies supported by this solver.

Public Attributes

- DFS
  Depth-first graph search.
- BFS
  Breadth-first graph search.
- UCS
  Uniform Cost Search ordered by accumulated route cost.
- GREEDY
  Greedy Best-First Search ordered by Manhattan distance.
- A_STAR
  A∗ ordered by route cost plus Manhattan distance.

9.1.1 Detailed Description

Search strategies supported by this solver.

9.1.2 Member Data Documentation

9.1.2.1 A_STAR

maze.search.MazeSolver.Algorithm.A_STAR

A∗ ordered by route cost plus Manhattan distance.

9.1.2.2 BFS

maze.search.MazeSolver.Algorithm.BFS

Breadth-first graph search.

9.1.2.3 DFS

maze.search.MazeSolver.Algorithm.DFS

Depth-first graph search.

9.1.2.4 GREEDY

maze.search.MazeSolver.Algorithm.GREEDY

Greedy Best-First Search ordered by Manhattan distance.

9.1.2.5 UCS

maze.search.MazeSolver.Algorithm.UCS

Uniform Cost Search ordered by accumulated route cost.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/search/MazeSolver.java

9.2 app.ApplicationPreferences Class Reference

Stores user-level application preferences that are needed before the JVM starts, such as the requested maximum heap size.

Static Public Member Functions

- static int getMaxHeapGb ()

  Returns the configured heap size in gigabytes, or 0 when the launcher should calculate a safe heap from current available memory.
- static void setMaxHeapGb (int heapGb) throws IOException

  Saves the requested maximum heap size.
- static Path preferencesPath ()

  Returns the location of the preferences file for diagnostics and launcher integration.

9.2.1 Detailed Description

Stores user-level application preferences that are needed before the JVM starts, such as the requested maximum heap size.

9.2.2 Member Function Documentation

9.2.2.1 getMaxHeapGb()

int app.ApplicationPreferences.getMaxHeapGb () [inline], [static]

Returns the configured heap size in gigabytes, or 0 when the launcher should calculate a safe heap from current available memory.

Returns

configured heap size in gigabytes, or zero for automatic sizing

9.2.2.2 preferencesPath()

Path app.ApplicationPreferences.preferencesPath () [inline], [static]

Returns the location of the preferences file for diagnostics and launcher integration.

Returns

user-level preferences file path

9.2.2.3 setMaxHeapGb()

void app.ApplicationPreferences.setMaxHeapGb (
int heapGb) throws IOException [inline], [static]

Saves the requested maximum heap size.

A value of 0 enables launch-time automatic sizing based on current available memory.

Parameters

heapGb	explicit heap size in gigabytes, or zero for automatic sizing

Exceptions

IOException	if the preferences file cannot be written

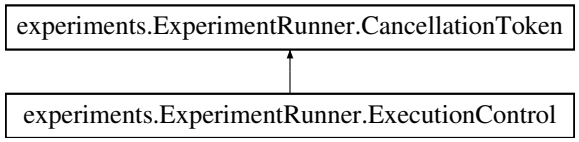
The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/app/ApplicationPreferences.java

9.3 experiments.ExperimentRunner.CancellationToken Interface Reference

Allows a caller to stop a long-running experiment.

Inheritance diagram for experiments.ExperimentRunner.CancellationToken:



Public Member Functions

- boolean isCancelled ()
  Indicates whether execution should stop before the next algorithm.

9.3.1 Detailed Description

Allows a caller to stop a long-running experiment.

9.3.2 Member Function Documentation

9.3.2.1 isCancelled()

boolean experiments.ExperimentRunner.CancellationToken.isCancelled ()

Indicates whether execution should stop before the next algorithm.

Returns

true when execution should be cancelled

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.4 experiments.TestSuite.CostMode Enum Reference

Movement-cost distribution used by an experiment row.

Public Attributes

- UNIFORM
  Assigns the same unit cost to every enabled movement.
- RANDOM
  Assigns seeded positive costs within the configured range.
- FROM_MAP
  Uses the directional movement costs embedded in a saved map.

9.4.1 Detailed Description

Movement-cost distribution used by an experiment row.

9.4.2 Member Data Documentation

9.4.2.1 FROM_MAP

experiments.TestSuite.CostMode.FROM_MAP

Uses the directional movement costs embedded in a saved map.

9.4.2.2 RANDOM

experiments.TestSuite.CostMode.RANDOM

Assigns seeded positive costs within the configured range.

9.4.2.3 UNIFORM

experiments.TestSuite.CostMode.UNIFORM

Assigns the same unit cost to every enabled movement.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.5 maze.map.Maze.Dir Enum Reference

Cardinal directions used for indexing node neighbors.

Public Member Functions

- Dir (int idx)
  Creates a direction enum value with a fixed neighbor-array index.
- int idx ()
  Gets the neighbor-array index associated with this direction.

Public Attributes

- NORTH =(0)
  Direction toward the previous row.
- EAST =(1)
  Direction toward the next column.
- SOUTH =(2)
  Direction toward the next row.
- WEST =(3)
  Direction toward the previous column.

9.5.1 Detailed Description

Cardinal directions used for indexing node neighbors.

9.5.2 Constructor & Destructor Documentation

9.5.2.1 Dir()

maze.map.Maze.Dir.Dir (
int idx) [inline]

Creates a direction enum value with a fixed neighbor-array index.

Parameters

idx	index used in a node's neighbor array

9.5.3 Member Function Documentation

9.5.3.1 idx()

int maze.map.Maze.Dir.idx () [inline]

Gets the neighbor-array index associated with this direction.

Returns

zero-based index for this direction in Node.neighbors

9.5.4 Member Data Documentation

9.5.4.1 EAST

maze.map.Maze.Dir.EAST =(1)

Direction toward the next column.

9.5.4.2 NORTH

maze.map.Maze.Dir.NORTH =(0)

Direction toward the previous row.

9.5.4.3 SOUTH

maze.map.Maze.Dir.SOUTH =(2)

Direction toward the next row.

9.5.4.4 WEST

maze.map.Maze.Dir.WEST =(3)

Direction toward the previous column.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java

9.6 experiments.TestSuite.EndpointMode Enum Reference

How an experiment row chooses its start and goal coordinates.

Public Attributes

- FIXED
  Uses manually validated start and goal coordinates.
- RANDOM
  Selects endpoints using the iteration's seeded random source.
- OPPOSITE_CORNERS
  Uses the two opposite interior corners of the map.
- APPROXIMATE_DISTANCE
  Selects endpoints near a requested Manhattan distance.

9.6.1 Detailed Description

How an experiment row chooses its start and goal coordinates.

9.6.2 Member Data Documentation

9.6.2.1 APPROXIMATE_DISTANCE

experiments.TestSuite.EndpointMode.APPROXIMATE_DISTANCE

Selects endpoints near a requested Manhattan distance.

9.6.2.2 FIXED

experiments.TestSuite.EndpointMode.FIXED

Uses manually validated start and goal coordinates.

9.6.2.3 OPPOSITE_CORNERS

experiments.TestSuite.EndpointMode.OPPOSITE_CORNERS

Uses the two opposite interior corners of the map.

9.6.2.4 RANDOM

experiments.TestSuite.EndpointMode.RANDOM

Selects endpoints using the iteration's seeded random source.

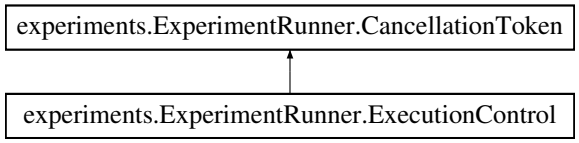
The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.7 experiments.ExperimentRunner.ExecutionControl Interface Reference

Adds cooperative pausing to cancellation for interactive runs.

Inheritance diagram for experiments.ExperimentRunner.ExecutionControl:



Public Member Functions

- boolean isPaused ()
  Indicates whether new work should wait for a resume request.
- void awaitIfPaused ()
  Waits until execution is resumed or cancellation is requested.

Public Member Functions inherited from experiments.ExperimentRunner.CancellationToken

- boolean isCancelled ()
  Indicates whether execution should stop before the next algorithm.

9.7.1 Detailed Description

Adds cooperative pausing to cancellation for interactive runs.

9.7.2 Member Function Documentation

9.7.2.1 awaitIfPaused()

void experiments.ExperimentRunner.ExecutionControl.awaitIfPaused ()

Waits until execution is resumed or cancellation is requested.

9.7.2.2 isPaused()

boolean experiments.ExperimentRunner.ExecutionControl.isPaused ()

Indicates whether new work should wait for a resume request.

Returns

true while execution is paused

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.8 experiments.TestSuite.ExecutionMode Enum Reference

Execution strategy for experiment solver runs.

Public Member Functions

- String toString ()

Public Attributes

- SEQUENTIAL
  Run solver algorithms one at a time.
- FIXED_PARALLEL
  Use a fixed bounded worker pool.
- ADAPTIVE_PARALLEL
  Select a bounded worker count from current CPU and heap headroom.

9.8.1 Detailed Description

Execution strategy for experiment solver runs.

9.8.2 Member Function Documentation

9.8.2.1 toString()

String experiments.TestSuite.ExecutionMode.toString () [inline]

9.8.3 Member Data Documentation

9.8.3.1 ADAPTIVE_PARALLEL

experiments.TestSuite.ExecutionMode.ADAPTIVE_PARALLEL

Select a bounded worker count from current CPU and heap headroom.

9.8.3.2 FIXED_PARALLEL

experiments.TestSuite.ExecutionMode.FIXED_PARALLEL

Use a fixed bounded worker pool.

9.8.3.3 SEQUENTIAL

experiments.TestSuite.ExecutionMode.SEQUENTIAL

Run solver algorithms one at a time.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.9 experiments.ExperimentCli Class Reference

Headless command-line entry point for running and benchmarking experiment suites without starting Swing.

Static Public Member Functions

- static void main (String[] args)
  Runs the command-line entry point.

9.9.1 Detailed Description

Headless command-line entry point for running and benchmarking experiment suites without starting Swing.

Example: java -cp build\classes experiments.ExperimentCli --suite suite.mstests
--mode FIXED_PARALLEL --repeat 3 --csv results.csv

9.9.2 Member Function Documentation

9.9.2.1 main()

void experiments.ExperimentCli.main (
String[] args) [inline], [static]

Runs the command-line entry point.

Parameters

args	command-line options

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentCli.java

9.10 experiments.ExperimentResult Class Reference

Immutable measurements from one algorithm run on one generated map.

Public Member Functions

- ExperimentResult (String suiteName, String configurationId, int iteration, long effectiveSeed, TestSuite.MapSource mapSource, String savedMapName, int rows, int columns, double requested↩ BlockedPercent, double actualBlockedPercent, int blockedCellCount, TestSuite.ReachabilityMode reachabilityMode, boolean generatedMapSolvable, TestSuite.CostMode costMode, int minimum↩ Cost, int maximumCost, int startRow, int startColumn, int goalRow, int goalColumn, int requestedManhattanDistance, int actualManhattanDistance, MazeSolver.Algorithm algorithm, boolean solved, int pathLength, long pathCost, int nodesExplored, long runtimeNs)

  Creates a result and validates fields that must never be negative.
- String getSuiteName ()

  Gets the suite name.
- String getConfigurationId ()

  Gets the test-row identifier.
- int getIteration ()

  Gets the one-based iteration number.
- long getEffectiveSeed ()

  Gets the effective map-generation seed.
- TestSuite.MapSource getMapSource ()

  Gets the map source used for this result.
- String getSavedMapName ()

  Gets the embedded saved-map filename.
- String getSeedLabel ()

  Gets a display value for the seed column.
- int getRows ()

  Gets the total map row count.
- int getColumns ()

  Gets the total map column count.

- double getRequestedBlockedPercent ()

  Gets the requested interior blocked percentage.
- double getActualBlockedPercent ()

  Gets the actual interior blocked percentage.
- int getBlockedCellCount ()

  Gets the actual blocked interior-cell count.
- TestSuite.ReachabilityMode getReachabilityMode ()

  Gets the configured reachability policy.
- boolean isGeneratedMapSolvable ()

  Indicates whether the generated map contains a route.
- TestSuite.CostMode getCostMode ()

  Gets the configured movement-cost mode.
- int getMinimumCost ()

  Gets the inclusive minimum movement cost.
- int getMaximumCost ()

  Gets the inclusive maximum movement cost.
- int getStartRow ()

  Gets the start row.
- int getStartColumn ()

  Gets the start column.
- int getGoalRow ()

  Gets the goal row.
- int getGoalColumn ()

  Gets the goal column.
- int getRequestedManhattanDistance ()

  Gets the requested Manhattan distance.
- int getActualManhattanDistance ()

  Gets the actual endpoint Manhattan distance.
- MazeSolver.Algorithm getAlgorithm ()

  Gets the measured algorithm.
- boolean isSolved ()

  Indicates whether the algorithm found a route.
- int getPathLength ()

  Gets the route length.
- long getPathCost ()

  Gets the total route cost.
- int getNodesExplored ()

  Gets the number of finalized or visited nodes.
- long getRuntimeNs ()

  Gets solver-only runtime in nanoseconds.
- double getRuntimeMs ()

  Gets runtime in milliseconds for display and CSV convenience.

9.10.1 Detailed Description

Immutable measurements from one algorithm run on one generated map.

9.10.2 Constructor & Destructor Documentation

9.10.2.1 ExperimentResult()

```
experiments.ExperimentResult.ExperimentResult (
                String suiteName,
                String configurationId,
                int iteration,
                long effectiveSeed,
                TestSuite.MapSource mapSource,
                String savedMapName,
                int rows,
                int columns,
                double requestedBlockedPercent,
                double actualBlockedPercent,
                int blockedCellCount,
                TestSuite.ReachabilityMode reachabilityMode,
                boolean generatedMapSolvable,
                TestSuite.CostMode costMode,
                int minimumCost,
                int maximumCost,
                int startRow,
                int startColumn,
                int goalRow,
                int goalColumn,
                int requestedManhattanDistance,
                int actualManhattanDistance,
                MazeSolver.Algorithm algorithm,
                boolean solved,
                int pathLength,
                long pathCost,
                int nodesExplored,
                long runtimeNs)    [inline]
```

Creates a result and validates fields that must never be negative.

Parameters

suiteName	suite that produced the result
configurationId	test-row identifier
iteration	one-based iteration number
effectiveSeed	seed used for this generated map
mapSource	source used to obtain the map
savedMapName	saved-map filename, or null for generated maps
rows	total map row count
columns	total map column count
requestedBlockedPercent	requested interior blocked percentage
actualBlockedPercent	actual interior blocked percentage
blockedCellCount	actual blocked interior-cell count
reachabilityMode	configured reachability policy
generatedMapSolvable	whether the generated map has a route
costMode	configured movement-cost mode

minimumCost	inclusive configured minimum movement cost
maximumCost	inclusive configured maximum movement cost
startRow	start row used by the solvers
startColumn	start column used by the solvers
goalRow	goal row used by the solvers
goalColumn	goal column used by the solvers
requestedManhattanDistance	requested endpoint distance, or -1 when not applicable
actualManhattanDistance	actual endpoint Manhattan distance
algorithm	algorithm measured by this result
solved	whether the algorithm found a route
pathLength	number of moves in the returned route, or -1 when unsolved
pathCost	total route cost, or -1 when unsolved
nodesExplored	number of finalized/visited nodes
runtimeNs	solver-only runtime in nanoseconds

Exceptions

IllegalArgumentException	if required text, modes, ranges, or metrics are invalid

9.10.3 Member Function Documentation

9.10.3.1 getActualBlockedPercent()

double experiments.ExperimentResult.getActualBlockedPercent () [inline]

Gets the actual interior blocked percentage.

Returns

actual interior blocked percentage

9.10.3.2 getActualManhattanDistance()

int experiments.ExperimentResult.getActualManhattanDistance () [inline]

Gets the actual endpoint Manhattan distance.

Returns

actual endpoint Manhattan distance

9.10.3.3 getAlgorithm()

MazeSolver.Algorithm experiments.ExperimentResult.getAlgorithm () [inline]

Gets the measured algorithm.

Returns

measured algorithm

9.10.3.4 getBlockedCellCount()

int experiments.ExperimentResult.getBlockedCellCount () [inline]

Gets the actual blocked interior-cell count.

Returns

actual blocked interior-cell count

9.10.3.5 getColumns()

int experiments.ExperimentResult.getColumns () [inline]

Gets the total map column count.

Returns

total map column count

9.10.3.6 getConfigurationId()

String experiments.ExperimentResult.getConfigurationId () [inline]

Gets the test-row identifier.

Returns

test-row identifier

9.10.3.7 getCostMode()

TestSuite.CostMode experiments.ExperimentResult.getCostMode () [inline]

Gets the configured movement-cost mode.

Returns

configured movement-cost mode

9.10.3.8 getEffectiveSeed()

long experiments.ExperimentResult.getEffectiveSeed () [inline]

Gets the effective map-generation seed.

Returns

effective map-generation seed

9.10.3.9 getGoalColumn()

int experiments.ExperimentResult.getGoalColumn () [inline]

Gets the goal column.

Returns

goal column

9.10.3.10 getGoalRow()

int experiments.ExperimentResult.getGoalRow () [inline]

Gets the goal row.

Returns

goal row

9.10.3.11 getIteration()

int experiments.ExperimentResult.getIteration () [inline]

Gets the one-based iteration number.

Returns

one-based iteration number

9.10.3.12 getMapSource()

TestSuite.MapSource experiments.ExperimentResult.getMapSource () [inline]

Gets the map source used for this result.

Returns

map source

9.10.3.13 getMaximumCost()

int experiments.ExperimentResult.getMaximumCost () [inline]

Gets the inclusive maximum movement cost.

Returns

inclusive maximum movement cost

9.10.3.14 getMinimumCost()

int experiments.ExperimentResult.getMinimumCost () [inline]

Gets the inclusive minimum movement cost.

Returns

inclusive minimum movement cost

9.10.3.15 getNodesExplored()

int experiments.ExperimentResult.getNodesExplored () [inline]

Gets the number of finalized or visited nodes.

Returns

number of finalized or visited nodes

9.10.3.16 getPathCost()

long experiments.ExperimentResult.getPathCost () [inline]

Gets the total route cost.

Returns

total route cost, or -1 when unsolved

9.10.3.17 getPathLength()

int experiments.ExperimentResult.getPathLength () [inline]

Gets the route length.

Returns

route length in moves, or -1 when unsolved

9.10.3.18 getReachabilityMode()

TestSuite.ReachabilityMode experiments.ExperimentResult.getReachabilityMode () [inline]

Gets the configured reachability policy.

Returns

configured reachability policy

9.10.3.19 getRequestedBlockedPercent()

double experiments.ExperimentResult.getRequestedBlockedPercent () [inline]

Gets the requested interior blocked percentage.

Returns

requested interior blocked percentage

9.10.3.20 getRequestedManhattanDistance()

int experiments.ExperimentResult.getRequestedManhattanDistance () [inline]

Gets the requested Manhattan distance.

Returns

requested distance, or -1 when not applicable

9.10.3.21 getRows()

int experiments.ExperimentResult.getRows () [inline]

Gets the total map row count.

Returns

total map row count

9.10.3.22 getRuntimeMs()

double experiments.ExperimentResult.getRuntimeMs () [inline]

Gets runtime in milliseconds for display and CSV convenience.

Returns

runtime in milliseconds

9.10.3.23 getRuntimeNs()

long experiments.ExperimentResult.getRuntimeNs () [inline]

Gets solver-only runtime in nanoseconds.

Returns

runtime in nanoseconds

9.10.3.24 getSavedMapName()

String experiments.ExperimentResult.getSavedMapName () [inline]

Gets the embedded saved-map filename.

Returns

filename, or null when unavailable

9.10.3.25 getSeedLabel()

String experiments.ExperimentResult.getSeedLabel () [inline]

Gets a display value for the seed column.

Returns

seed value, or N/A for an embedded saved map

9.10.3.26 getStartColumn()

int experiments.ExperimentResult.getStartColumn () [inline]

Gets the start column.

Returns

start column

9.10.3.27 getStartRow()

int experiments.ExperimentResult.getStartRow () [inline]

Gets the start row.

Returns

start row

9.10.3.28 getSuiteName()

String experiments.ExperimentResult.getSuiteName () [inline]

Gets the suite name.

Returns

suite name

9.10.3.29 isGeneratedMapSolvable()

boolean experiments.ExperimentResult.isGeneratedMapSolvable () [inline]

Indicates whether the generated map contains a route.

Returns

true when the generated map contains a route

9.10.3.30 isSolved()

boolean experiments.ExperimentResult.isSolved () [inline]

Indicates whether the algorithm found a route.

Returns

true when the algorithm found a route

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentResult.java

9.11 experiments.ExperimentRunner Class Reference

Executes configured experiment rows without Swing or rendering overhead.

Classes

- interface ProgressListener
  Receives progress after each completed algorithm run.
- interface MapProgressListener
  Receives progress after each map has finished generating.
- interface SolveProgressListener
  Receives throttled progress from solver workers while they are active.
- class SolveProgress
  Immutable diagnostic snapshot for one active solver worker.
- interface CancellationToken
  Allows a caller to stop a long-running experiment.
- interface ExecutionControl
  Adds cooperative pausing to cancellation for interactive runs.
- enum TaskPriority

Static Public Member Functions

- static List< ExperimentResult > run (TestSuite suite)

  Runs a complete suite.
- static List< ExperimentResult > run (TestSuite suite, ProgressListener listener, CancellationToken cancellation)

  Runs a suite with progress and cancellation support.
- static List< ExperimentResult > run (TestSuite suite, ProgressListener listener, CancellationToken cancellation, SolveProgressListener solveProgress)

  Runs a suite with completed-result and throttled solver diagnostics.
- static List< ExperimentResult > run (TestSuite suite, ProgressListener listener, CancellationToken cancellation, SolveProgressListener solveProgress, MapProgressListener mapProgress)

  Runs a suite with solver and generated-map progress diagnostics.
- static int getParallelWorkerCount ()

  Gets the configured worker capacity for the currently running suite.
- static int getAdaptiveWorkerCount ()

  Selects workers conservatively from available heap and system CPU load.
- static int getActiveWorkerCount ()

  Gets the worker capacity currently assigned to the active suite.
- static int getActiveMapWorkerCount ()

  Gets the currently executing map-generation worker count.
- static int getActiveSolverWorkerCount ()

  Gets the currently executing solver worker count.
- static int getIntervalPeakMapWorkerCount ()

  Returns and resets the maximum map-worker count observed since the previous call.
- static int getIntervalPeakSolverWorkerCount ()

  Returns and resets the maximum solver-worker count observed since the previous call.
- static String formatSummary (List< ExperimentResult > results)

  Formats a compact terminal summary for completed results.
- static void exportMaps (TestSuite suite, Path directory) throws IOException

  Exports every map used by a suite, including one file per generated iteration and one file per saved-map row.

Static Public Attributes

- static final CancellationToken NEVER_CANCEL = () -> false

  Token that never cancels execution.

9.11.1 Detailed Description

Executes configured experiment rows without Swing or rendering overhead.

One map is generated per row iteration and reused by every selected algorithm. Solver timing therefore excludes map generation, result construction, callbacks, and console output. Parallel execution uses bounded admission so queued solver work cannot retain an unbounded number of generated maps.

9.11.2 Member Function Documentation

9.11.2.1 exportMaps()

void experiments.ExperimentRunner.exportMaps (
TestSuite suite,
Path directory) throws IOException [inline], [static]

Exports every map used by a suite, including one file per generated iteration and one file per saved-map row.

Parameters

suite	validated suite
directory	parent directory in which the suite map folder is created

Exceptions

IOException	if a map cannot be written

9.11.2.2 formatSummary()

String experiments.ExperimentRunner.formatSummary (
List< ExperimentResult > results) [inline], [static]

Formats a compact terminal summary for completed results.

Parameters

results	results to summarize

Returns

tabular summary text

9.11.2.3 getActiveMapWorkerCount()

int experiments.ExperimentRunner.getActiveMapWorkerCount () [inline], [static]

Gets the currently executing map-generation worker count.

Returns

currently executing map-generation workers

9.11.2.4 getActiveSolverWorkerCount()

int experiments.ExperimentRunner.getActiveSolverWorkerCount () [inline], [static]

Gets the currently executing solver worker count.

Returns

currently executing solver workers

9.11.2.5 getActiveWorkerCount()

int experiments.ExperimentRunner.getActiveWorkerCount () [inline], [static]

Gets the worker capacity currently assigned to the active suite.

Returns

worker capacity, one for sequential mode, or zero while idle

9.11.2.6 getAdaptiveWorkerCount()

int experiments.ExperimentRunner.getAdaptiveWorkerCount () [inline], [static]

Selects workers conservatively from available heap and system CPU load.

Returns

adaptive worker count for the next map

9.11.2.7 getIntervalPeakMapWorkerCount()

int experiments.ExperimentRunner.getIntervalPeakMapWorkerCount () [inline], [static]

Returns and resets the maximum map-worker count observed since the previous call.

Returns

interval peak map-worker count

9.11.2.8 getIntervalPeakSolverWorkerCount()

int experiments.ExperimentRunner.getIntervalPeakSolverWorkerCount () [inline], [static]

Returns and resets the maximum solver-worker count observed since the previous call.

Returns

interval peak solver-worker count

9.11.2.9 getParallelWorkerCount()

int experiments.ExperimentRunner.getParallelWorkerCount () [inline], [static]

Gets the configured worker capacity for the currently running suite.

Returns

worker capacity, one for sequential mode, or zero while idle

9.11.2.10 run() [1/4]

List< ExperimentResult > experiments.ExperimentRunner.run (
TestSuite suite) [inline], [static]

Runs a complete suite.

Parameters

suite	validated suite to execute

Returns

immutable results in row, iteration, and algorithm order

Exceptions

IllegalArgumentException	if the suite is null or invalid

9.11.2.11 run() [2/4]

List< ExperimentResult > experiments.ExperimentRunner.run (
TestSuite suite,
ProgressListener listener,
CancellationToken cancellation) [inline], [static]

Runs a suite with progress and cancellation support.

When cancellation is requested, completed results are returned and no partially executed solver is converted into a result.

Parameters

suite	validated suite to execute
listener	optional progress listener
cancellation	optional cancellation token

Returns

immutable completed results

Exceptions

IllegalArgumentException	if the suite is null or invalid

9.11.2.12 run() [3/4]

List< ExperimentResult > experiments.ExperimentRunner.run (
TestSuite suite,
ProgressListener listener,
CancellationToken cancellation,
SolveProgressListener solveProgress) [inline], [static]

Runs a suite with completed-result and throttled solver diagnostics.

Parameters

suite	validated suite to execute
listener	optional completed-result listener
cancellation	optional cancellation token
solveProgress	optional active-solver listener

Returns

immutable completed results

9.11.2.13 run() [4/4]

List< ExperimentResult > experiments.ExperimentRunner.run (
TestSuite suite,
ProgressListener listener,
CancellationToken cancellation,
SolveProgressListener solveProgress,
MapProgressListener mapProgress) [inline], [static]

Runs a suite with solver and generated-map progress diagnostics.

Parameters

suite	validated suite to execute
listener	optional completed-result listener
cancellation	optional cancellation token
solveProgress	optional active-solver listener
mapProgress	optional generated-map listener

Returns

immutable completed results

9.11.3 Member Data Documentation

9.11.3.1 NEVER_CANCEL

final CancellationToken experiments.ExperimentRunner.NEVER_CANCEL = () -> false [static]

Token that never cancels execution.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.12 maze.map.Maze.GeneratedExperimentMap Class Reference

Metadata for one generated experiment map.

Public Member Functions

- Maze getMaze ()
  Gets the generated maze containing the selected endpoints and costs.
- long getEffectiveSeed ()
  Gets the effective seed used to generate this map.
- int getBlockedCellCount ()
  Gets the blocked interior-cell count.
- double getActualBlockedPercent ()
  Gets the actual blocked percentage among interior cells.
- boolean isSolvable ()
  Indicates whether the selected endpoints are connected.
- int getStartRow ()
  Gets the selected start row.
- int getStartColumn ()
  Gets the selected start column.
- int getGoalRow ()
  Gets the selected goal row.
- int getGoalColumn ()
  Gets the selected goal column.
- int getActualManhattanDistance ()
  Gets the actual Manhattan distance between endpoints.

Static Public Member Functions

- static GeneratedExperimentMap fromSavedMap (Maze maze, long seed)
  Wraps an imported maze as an experiment map without regenerating it.

9.12.1 Detailed Description

Metadata for one generated experiment map.

The wrapped maze is shared read-only by the solver jobs for one iteration. Each job creates its own MazeSolver, so search state is never shared between algorithms.

9.12.2 Member Function Documentation

9.12.2.1 fromSavedMap()

GeneratedExperimentMap maze.map.Maze.GeneratedExperimentMap.fromSavedMap (
Maze maze,
long seed) [inline], [static]

Wraps an imported maze as an experiment map without regenerating it.

Parameters

maze	imported maze to execute
seed	effective seed associated with the experiment row

Returns

generated-map wrapper containing the imported maze

9.12.2.2 getActualBlockedPercent()

double maze.map.Maze.GeneratedExperimentMap.getActualBlockedPercent () [inline]

Gets the actual blocked percentage among interior cells.

Returns

actual blocked percentage

9.12.2.3 getActualManhattanDistance()

int maze.map.Maze.GeneratedExperimentMap.getActualManhattanDistance () [inline]

Gets the actual Manhattan distance between endpoints.

Returns

actual Manhattan distance

9.12.2.4 getBlockedCellCount()

int maze.map.Maze.GeneratedExperimentMap.getBlockedCellCount () [inline]

Gets the blocked interior-cell count.

Returns

blocked interior-cell count

9.12.2.5 getEffectiveSeed()

long maze.map.Maze.GeneratedExperimentMap.getEffectiveSeed () [inline]

Gets the effective seed used to generate this map.

Returns

effective seed

9.12.2.6 getGoalColumn()

int maze.map.Maze.GeneratedExperimentMap.getGoalColumn () [inline]

Gets the selected goal column.

Returns

goal column

9.12.2.7 getGoalRow()

int maze.map.Maze.GeneratedExperimentMap.getGoalRow () [inline]

Gets the selected goal row.

Returns

goal row

9.12.2.8 getMaze()

Maze maze.map.Maze.GeneratedExperimentMap.getMaze () [inline]

Gets the generated maze containing the selected endpoints and costs.

Returns

generated maze

9.12.2.9 getStartColumn()

int maze.map.Maze.GeneratedExperimentMap.getStartColumn () [inline]

Gets the selected start column.

Returns

start column

9.12.2.10 getStartRow()

int maze.map.Maze.GeneratedExperimentMap.getStartRow () [inline]

Gets the selected start row.

Returns

start row

9.12.2.11 isSolvable()

boolean maze.map.Maze.GeneratedExperimentMap.isSolvable () [inline]

Indicates whether the selected endpoints are connected.

Returns

true when the generated map has a route

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java

9.13 experiments.ExperimentRunner.MapProgressListener Interface Reference

Receives progress after each map has finished generating.

Public Member Functions

- void onProgress (int generatedMaps, int totalMaps, long reservedBytes, long admissionBudget↩ Bytes, long heapUsedBytes, long maxHeapBytes, int loadedMaps, String message)
  Reports generated map progress.

9.13.1 Detailed Description

Receives progress after each map has finished generating.

9.13.2 Member Function Documentation

9.13.2.1 onProgress()

```
void experiments.ExperimentRunner.MapProgressListener.onProgress (
            int generatedMaps,
            int totalMaps,
            long reservedBytes,
            long admissionBudgetBytes,
            long heapUsedBytes,
            long maxHeapBytes,
            int loadedMaps,
            String message)
```

Reports generated map progress.

Parameters

generatedMaps	maps that have finished generating
totalMaps	total maps planned for the suite
reservedBytes	estimated bytes reserved by admitted maps
admissionBudgetBytes	current admission budget
heapUsedBytes	current JVM heap usage
maxHeapBytes	configured JVM maximum heap
loadedMaps	generated maps waiting for solver dispatch
message	formatted lifecycle message for the map log

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.14 experiments.TestSuite.MapSource Enum Reference

Source used to obtain the map for an experiment row.

Public Attributes

- AUTOMATIC_SEED
  Generate one random seed for the test-case execution or save.
- MANUAL_SEED
  Generate maps from the explicitly entered seed.
- SAVED_MAP
  Reuse the embedded serialized map.

9.14.1 Detailed Description

Source used to obtain the map for an experiment row.

9.14.2 Member Data Documentation

9.14.2.1 AUTOMATIC_SEED

experiments.TestSuite.MapSource.AUTOMATIC_SEED

Generate one random seed for the test-case execution or save.

9.14.2.2 MANUAL_SEED

experiments.TestSuite.MapSource.MANUAL_SEED

Generate maps from the explicitly entered seed.

9.14.2.3 SAVED_MAP

experiments.TestSuite.MapSource.SAVED_MAP

Reuse the embedded serialized map.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.15 maze.map.Maze Class Reference

Core maze model storing occupancy, neighbor links, and start/goal nodes.

Classes

- enum Dir
  Cardinal directions used for indexing node neighbors.
- class Node
  Represents a single cell in the maze grid.
- class GeneratedExperimentMap
  Metadata for one generated experiment map.
- class NeighborValidationResult
  Validation result for neighbor consistency checks.

Public Member Functions

- Maze (int length, int width)

  Initializes a maze with fixed dimensions.
- void createCell (int row, int col, int direction)

  Marks a cell as occupied and creates a node at that coordinate.
- void exportToFile (String filePath) throws IOException

  Export the current maze to text format for import/export.
- String exportToText ()

  Exports the current maze using the complete saveable map format.
- int getMoveCost (Node from, Node to)

  Gets the cost of moving between two adjacent nodes.
- boolean canMove (Node from, Dir direction)

  Indicates whether movement is enabled from a node in a direction.
- void setStart (int row, int col)

  Sets the start node if the given coordinate is occupied.
- void setGoal (int row, int col)

  Sets the goal node and updates the goal flag on nodes accordingly.
- Node getStart ()

  Returns the current start node.
- Node getGoal ()

  Returns the current goal node.
- Node getNode (int row, int col)

  Returns the node at a coordinate if that cell is occupied.
- boolean isInBounds (int row, int col)

  Checks whether coordinates are inside maze boundaries.
- boolean isOccupied (int row, int col)

  Checks whether a cell is a traversable/occupied cell.
- int getLength ()

  Gets the maze row count.
- int getWidth ()

  Gets the maze column count.
- NeighborValidationResult validateNeighborConnections (int maxIssuesToCollect)

  Validate all node-to-neighbor links against occupied adjacency.
- List< String > buildNeighborTableLines ()

  Build a compact, line-by-line neighbor table for occupied cells.

Static Public Member Functions

- static Maze importFromFile (String filePath) throws IOException

  Import a maze from text using this format: line 1: row count line 2: column count line 3: start: row,col line 4: end: row,col line 5+: layout rows of 1s and 0s optional: WEIGHTS followed by length * width rows of N,E,S,W costs.
- static Maze importFromText (String contents) throws IOException

  Imports a maze from the complete text representation.
- static Maze generateRandomMaze (int length, int width)

  Builds a random, guaranteed-solvable maze with the given dimensions.
- static Maze generateRandomMaze (int length, int width, boolean weighted, int minWeight, int maxWeight)

  Builds a random, guaranteed-solvable maze with optional directional weights.

- static Maze generateRandomMaze (int length, int width, boolean weighted, int minWeight, int maxWeight, boolean sameCostBothDirections)

  Builds a random maze with optional weighted movement and reverse-edge matching.
- static GeneratedExperimentMap generateExperimentMap (TestSuite.TestCase testCase, long effectiveSeed)

  Generates a seeded experiment map from a validated test configuration.

Static Public Attributes

- static final String MAP_EXTENSION = ".msmap"

  Extension used for editable maze map files.
- static final int MIN_MOVEMENT_COST = 1

  Minimum allowed positive movement cost.
- static final int MAX_MOVEMENT_COST = 100

  Maximum allowed movement cost for all maps and generated mazes.
- static final int DISABLED_MOVEMENT_COST = 0

  Cost value reserved for a disabled or non-existent movement direction.

9.15.1 Detailed Description

Core maze model storing occupancy, neighbor links, and start/goal nodes.

The maze uses a rectangular grid of Node objects. Traversable cells contain a Node reference and blocked cells are null. Neighbor links represent geometric adjacency between traversable cells, while each direction has a movement cost. Enabled movement costs are positive integers from MIN_MOVEMENT_COST through MAX_MOVEMENT_COST; zero is reserved for disabled or non-existent movement. Direction order is always [NORTH, EAST, SOUTH, WEST].

Example import format for importFromFile(String) is: row count, column count, start: row,col, end: row,col, then one layout row per maze row using 1 (path) and 0 (wall). An optional WEIGHTS section may follow the layout. It contains one comma-separated four-value row for every grid coordinate in row-major order. Values use [N,E,S,W] order; 0 means no permitted movement and positive integer values are movement costs. If the section is absent, adjacent traversable cells default to cost 1. Example:

```
4
5
start: 3,0
end: 0,4
11111
10001
10101
11111
```

Typical implementation flow:

```
Maze maze = Maze.importFromFile("test/resources/maps/test_346x346.msmap");
Maze.NeighborValidationResult vr = maze.validateNeighborConnections(25);
if (!vr.isValid()) {
for (String issue : vr.getIssues()) {
System.out.println(issue);
}
}
maze.exportToFile("out.msmap");
```

9.15.2 Constructor & Destructor Documentation

9.15.2.1 Maze()

maze.map.Maze.Maze (
int length,
int width) [inline]

Initializes a maze with fixed dimensions.

A new maze starts empty: all cells are null (blocked) until paths are created explicitly via createCell(int, int, int) or populated through importFromFile(String).

Parameters

length	total number of rows in the maze
width	total number of columns in the maze

9.15.3 Member Function Documentation

9.15.3.1 buildNeighborTableLines()

List< String > maze.map.Maze.buildNeighborTableLines () [inline]

Build a compact, line-by-line neighbor table for occupied cells.

Returns

list of formatted lines describing neighbors for debugging

9.15.3.2 canMove()

boolean maze.map.Maze.canMove (
Node from,
Dir direction) [inline]

Indicates whether movement is enabled from a node in a direction.

Parameters

from	source node
direction	direction to inspect

Returns

true when a neighbor exists and its movement cost is positive

9.15.3.3 createCell()

void maze.map.Maze.createCell (
int row,
int col,
int direction) [inline]

Marks a cell as occupied and creates a node at that coordinate.

Establishes bidirectional neighbor links immediately: this node is linked to existing occupied neighbors, and those neighbors are updated to point back. The direction parameter is currently reserved and ignored.

Parameters

row	row index of the cell
col	column index of the cell
direction	reserved parameter for future directional creation logic

9.15.3.4 exportToFile()

void maze.map.Maze.exportToFile (
String filePath) throws IOException [inline]

Export the current maze to text format for import/export.

The output follows the same schema accepted by importFromFile(String).

Parameters

filePath	destination file path

Exceptions

IOException	if writing fails
IllegalStateException	if start or end is not set

9.15.3.5 exportToText()

String maze.map.Maze.exportToText () [inline]

Exports the current maze using the complete saveable map format.

Returns

serialized maze text

9.15.3.6 generateExperimentMap()

GeneratedExperimentMap maze.map.Maze.generateExperimentMap (
TestSuite.TestCase testCase,
long effectiveSeed) [inline], [static]

Generates a seeded experiment map from a validated test configuration.

The outside border is always blocked and is excluded from the requested density calculation. The interior starts as a seeded, one-cell-wide traditional corridor maze. The requested density is then applied by opening additional cells or removing suitable dead ends, preserving the corridor structure instead of producing independent cell noise. The generated map uses one effective seed for its layout, endpoint selection, and movement costs. Algorithms should run against the returned maze without regenerating it.

Parameters

testCase	validated experiment configuration
effectiveSeed	seed resolved for the containing test-case row

Returns

generated map and metadata used to describe the instance

Exceptions

IllegalArgumentException	if the configuration is invalid or a valid endpoint pair cannot be selected

9.15.3.7 generateRandomMaze() [1/3]

Maze maze.map.Maze.generateRandomMaze (
int length,
int width) [inline], [static]

Builds a random, guaranteed-solvable maze with the given dimensions.

The generated maze keeps the outer border blocked and carves interior corridors using a depth-first backtracking strategy to produce a classic maze appearance. For even dimensions, a follow-up pass extends corridors into the final interior row/column so right/bottom edges are not left as persistent solid strips.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions are smaller than 3

9.15.3.8 generateRandomMaze() [2/3]

Maze maze.map.Maze.generateRandomMaze (
int length,
int width,
boolean weighted,
int minWeight,
int maxWeight) [inline], [static]

Builds a random, guaranteed-solvable maze with optional directional weights.

When weighting is enabled, each permitted connection receives a random positive integer cost in the inclusive range [minWeight, maxWeight]. The five-argument overload uses the same cost in both directions by default.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)
weighted	whether to assign random movement costs
minWeight	inclusive minimum movement cost
maxWeight	inclusive maximum movement cost

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions or weight bounds are invalid

9.15.3.9 generateRandomMaze() [3/3]

Maze maze.map.Maze.generateRandomMaze (
int length,
int width,
boolean weighted,
int minWeight,
int maxWeight,
boolean sameCostBothDirections) [inline], [static]

Builds a random maze with optional weighted movement and reverse-edge matching.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)
weighted	whether to assign random movement costs
minWeight	inclusive minimum movement cost
maxWeight	inclusive maximum movement cost
sameCostBothDirections	whether each connection uses one cost in both directions

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions or weight bounds are invalid

9.15.3.10 getGoal()

Node maze.map.Maze.getGoal () [inline]

Returns the current goal node.

Returns

goal node, or null if not set

9.15.3.11 getLength()

int maze.map.Maze.getLength () [inline]

Gets the maze row count.

Returns

total number of rows

9.15.3.12 getMoveCost()

int maze.map.Maze.getMoveCost (
Node from,
Node to) [inline]

Gets the cost of moving between two adjacent nodes.

Parameters

from	source node
to	destination node

Returns

zero when movement is disabled, otherwise the positive cost

Exceptions

IllegalArgumentException	if the nodes are not adjacent

9.15.3.13 getNode()

Node maze.map.Maze.getNode (
int row,
int col) [inline]

Returns the node at a coordinate if that cell is occupied.

Parameters

row	row coordinate to query
col	column coordinate to query

Returns

node at the coordinate, or null if not occupied

9.15.3.14 getStart()

Node maze.map.Maze.getStart () [inline]

Returns the current start node.

Returns

start node, or null if not set

9.15.3.15 getWidth()

int maze.map.Maze.getWidth () [inline]

Gets the maze column count.

Returns

total number of columns

9.15.3.16 importFromFile()

Maze maze.map.Maze.importFromFile (
String filePath) throws IOException [inline], [static]

Import a maze from text using this format: line 1: row count line 2: column count line 3: start: row,col line 4: end: row,col line 5+: layout rows of 1s and 0s optional: WEIGHTS followed by length ∗ width rows of N,E,S,W costs.

Cells marked 1 are created through createCell(int, int, int) so neighbor links are built incrementally and remain bidirectionally consistent.

Parameters

filePath	path to the maze text file

Returns

newly created Maze instance populated from file data

Exceptions

IOException	if file reading fails
IllegalArgumentException	if file contents are invalid

9.15.3.17 importFromText()

Maze maze.map.Maze.importFromText (
String contents) throws IOException [inline], [static]

Imports a maze from the complete text representation.

Parameters

contents	map file contents

Returns

imported maze

Exceptions

IOException	if the text cannot be read

9.15.3.18 isInBounds()

boolean maze.map.Maze.isInBounds (
int row,
int col) [inline]

Checks whether coordinates are inside maze boundaries.

Parameters

row	row coordinate to test
col	column coordinate to test

Returns

true if in range, otherwise false

9.15.3.19 isOccupied()

boolean maze.map.Maze.isOccupied (
int row,
int col) [inline]

Checks whether a cell is a traversable/occupied cell.

Parameters

row	row coordinate to test
col	column coordinate to test

Returns

true if in bounds and occupied, otherwise false

9.15.3.20 setGoal()

void maze.map.Maze.setGoal (
int row,
int col) [inline]

Sets the goal node and updates the goal flag on nodes accordingly.

Parameters

row	goal row coordinate
col	goal column coordinate

9.15.3.21 setStart()

void maze.map.Maze.setStart (
int row,
int col) [inline]

Sets the start node if the given coordinate is occupied.

Parameters

row	start row coordinate
col	start column coordinate

9.15.3.22 validateNeighborConnections()

NeighborValidationResult maze.map.Maze.validateNeighborConnections (
int maxIssuesToCollect) [inline]

Validate all node-to-neighbor links against occupied adjacency.

Parameters

maxIssuesToCollect	maximum number of issue messages to store.

Returns

validation result containing status, issue count, and messages

9.15.4 Member Data Documentation

9.15.4.1 DISABLED_MOVEMENT_COST

final int maze.map.Maze.DISABLED_MOVEMENT_COST = 0 [static]

Cost value reserved for a disabled or non-existent movement direction.

9.15.4.2 MAP_EXTENSION

final String maze.map.Maze.MAP_EXTENSION = ".msmap" [static]

Extension used for editable maze map files.

9.15.4.3 MAX_MOVEMENT_COST

final int maze.map.Maze.MAX_MOVEMENT_COST = 100 [static]

Maximum allowed movement cost for all maps and generated mazes.

9.15.4.4 MIN_MOVEMENT_COST

final int maze.map.Maze.MIN_MOVEMENT_COST = 1 [static]

Minimum allowed positive movement cost.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java

9.16 maze.map.MazeFileIO Class Reference

Reads and writes the human-readable .msmap format.

Static Public Member Functions

- static Maze importFromFile (String filePath) throws IOException
  Imports a map file.
- static Maze importFromText (String contents) throws IOException
  Imports a complete map text representation.
- static void exportToFile (Maze maze, String filePath) throws IOException
  Exports a maze to a checked .msmap path.
- static String exportToText (Maze maze)
  Serializes a maze using the complete weighted map format.

9.16.1 Detailed Description

Reads and writes the human-readable .msmap format.

This class is deliberately separate from Maze: the maze remains the domain model while this class owns file-format concerns. The static convenience methods on Maze delegate here for source compatibility.

9.16.2 Member Function Documentation

9.16.2.1 exportToFile()

void maze.map.MazeFileIO.exportToFile (
Maze maze,
String filePath) throws IOException [inline], [static]

Exports a maze to a checked .msmap path.

Parameters

maze	maze to export
filePath	destination path

Exceptions

IOException	if validation or writing fails

9.16.2.2 exportToText()

String maze.map.MazeFileIO.exportToText (
Maze maze) [inline], [static]

Serializes a maze using the complete weighted map format.

Parameters

maze	maze to serialize

Returns

map text

9.16.2.3 importFromFile()

Maze maze.map.MazeFileIO.importFromFile (
String filePath) throws IOException [inline], [static]

Imports a map file.

Parameters

filePath	source path

Returns

imported maze

Exceptions

IOException	if the file cannot be read

9.16.2.4 importFromText()

Maze maze.map.MazeFileIO.importFromText (
String contents) throws IOException [inline], [static]

Imports a complete map text representation.

Parameters

contents	map text

Returns

imported maze

Exceptions

IOException	retained for compatibility with the legacy API

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/MazeFileIO.java

9.17 ui.MazeRenderer Class Reference

Renderer and UI controller for the maze application.

Classes

- enum WeightDisplayState
  Describes how a weight label should be styled based on the solve state.

Static Public Member Functions

- static void launchFromTitleScreen ()

  Starts the renderer-driven application flow from the title screen.
- static void shutdownApplication ()

  Stops active work, closes all application windows, and terminates the JVM.
- static void reviewExperiment (Maze maze, MazeSolver.Algorithm algorithm)

  Opens an animated review session for an experiment result.
- static void reviewExperiment (Maze maze, MazeSolver.Algorithm algorithm, Runnable onBack)

  Opens an experiment result preview with a callback for returning to its owning screen.
- static boolean setNodeColor (Maze.Node node, Color color)

  Sets one node color using maze coordinates.
- static boolean resetNodeColor (Maze.Node node)

  Restores one node to its default semantic color.

9.17.1 Detailed Description

Renderer and UI controller for the maze application.

This class owns the full Swing-driven UX flow:

- Title screen mode selection (import or random generation)
- Maze file selection or random maze creation
- Maze grid rendering with a paint-based renderer for all maze sizes
- Runtime solver controls (start/pause/restart, delay, instant mode, map load/save/generate)
- Zoom handling (buttons and Ctrl+mouse wheel)
- Optional directional movement-cost overlay

Typical usage is through launchFromTitleScreen() from the app entrypoint. Example:

```
SwingUtilities.invokeLater(MazeRenderer::launchFromTitleScreen);
```

Typical extension point for custom highlighting:

```
Maze.Node node = maze.getStart();
MazeRenderer.setNodeColor(node, Color.MAGENTA);
MazeRenderer.resetNodeColor(node);
```

9.17.2 Member Function Documentation

9.17.2.1 launchFromTitleScreen()

void ui.MazeRenderer.launchFromTitleScreen () [inline], [static]

Starts the renderer-driven application flow from the title screen.

This is the recommended public entrypoint for GUI startup.

9.17.2.2 resetNodeColor()

boolean ui.MazeRenderer.resetNodeColor (
Maze.Node node) [inline], [static]

Restores one node to its default semantic color.

Default semantic color rules are: start=green, goal=blue, traversable=white, blocked=dark gray.

Parameters

node	maze node to reset

Returns

true when update was accepted

9.17.2.3 reviewExperiment() [1/2]

void ui.MazeRenderer.reviewExperiment (
Maze maze,
MazeSolver.Algorithm algorithm) [inline], [static]

Opens an animated review session for an experiment result.

Parameters

maze	exact map used by the result
algorithm	algorithm to select initially

9.17.2.4 reviewExperiment() [2/2]

void ui.MazeRenderer.reviewExperiment (
Maze maze,
MazeSolver.Algorithm algorithm,
Runnable onBack) [inline], [static]

Opens an experiment result preview with a callback for returning to its owning screen.

Parameters

maze	exact map used by the result
algorithm	algorithm to select initially
onBack	callback invoked when the preview is closed

9.17.2.5 setNodeColor()

boolean ui.MazeRenderer.setNodeColor (
Maze.Node node,
Color color) [inline], [static]

Sets one node color using maze coordinates.

Typical usage is temporary overlays (for example hints or debug markers) on top of solver coloring.

Parameters

node	maze node to color
color	target color

Returns

true when update was accepted

9.17.2.6 shutdownApplication()

void ui.MazeRenderer.shutdownApplication () [inline], [static]

Stops active work, closes all application windows, and terminates the JVM.

This is used by every top-level window so closing the UI cannot leave an experiment worker or solve session running in the background.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/ui/MazeRenderer.java

9.18 maze.search.MazeSolver Class Reference

Incremental maze solver that supports DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗.

Classes

- enum Algorithm
  Search strategies supported by this solver.

Public Member Functions

- MazeSolver ()
  Creates an unconfigured solver using DFS as its default algorithm.
- void setAlg (String alg)
  Sets which algorithm to run.
- String getAlg ()
  Gets the active algorithm label.
- void setMaze (Maze maze)
  Assigns the maze instance to solve.
- Maze.Node updateSolve ()
  Advances solving by exactly one visited node.
- boolean isFinished ()
  Indicates whether this run has terminated.
- boolean isSolved ()
  Indicates whether the current run found the goal.
- int getVisitedCount ()
  Gets count of nodes visited in the current run.
- long getSolveTimeNs ()
  Gets the total computation time elapsed in the current run.
- long getPathCost ()
  Gets the total movement cost of the final path.
- long getCurrentPathCost ()
  Gets the cost of the parent route for the most recently visited node.
- List< Point > getFinalPath ()
  Gets the final path after a successful solve.
- void resetSearch ()
  Resets transient search state so solving can start fresh.

9.18.1 Detailed Description

Incremental maze solver that supports DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗.

The solver keeps internal state between calls to updateSolve(), so callers can visualize or inspect each step one node at a time. Reported solve time measures the elapsed time of each search-step call, excluding timer delays and UI repaint work.

Typical usage: create a solver, set algorithm and maze, then call updateSolve() repeatedly until isFinished() returns true. Example:

```
MazeSolver solver = new MazeSolver();
solver.setMaze(maze);
solver.setAlg("A*");
while (!solver.isFinished()) {
solver.updateSolve();
}
if (solver.isSolved()) {
List<Point> path = solver.getFinalPath();
}
```

9.18.2 Constructor & Destructor Documentation

9.18.2.1 MazeSolver()

maze.search.MazeSolver.MazeSolver () [inline]

Creates an unconfigured solver using DFS as its default algorithm.

9.18.3 Member Function Documentation

9.18.3.1 getAlg()

String maze.search.MazeSolver.getAlg () [inline]

Gets the active algorithm label.

Returns

display label for the current algorithm

9.18.3.2 getCurrentPathCost()

long maze.search.MazeSolver.getCurrentPathCost () [inline]

Gets the cost of the parent route for the most recently visited node.

This is the current branch cost, not the sum of all explored branches. It is updated in constant time as each node is visited and remains useful for displaying progress during an incomplete search.

Returns

current route cost, or -1 before the first search step

9.18.3.3 getFinalPath()

List< Point > maze.search.MazeSolver.getFinalPath () [inline]

Gets the final path after a successful solve.

The path is traced backward using the internal parent table, so the returned list begins at the goal and ends at the start.

Typical UI code can iterate as-is for reverse painting, or reverse the list first if start-to-goal order is preferred.

Returns

goal-to-start path points where x=col, y=row; empty if unsolved

9.18.3.4 getPathCost()

long maze.search.MazeSolver.getPathCost () [inline]

Gets the total movement cost of the final path.

This is the sum of the directional movement costs in the path. It is -1 before a path is found or when the search is unsuccessful.

Returns

final path cost, or -1 when no path is available

9.18.3.5 getSolveTimeNs()

long maze.search.MazeSolver.getSolveTimeNs () [inline]

Gets the total computation time elapsed in the current run.

Returns

elapsed computation time in nanoseconds, excluding timer and UI delays

9.18.3.6 getVisitedCount()

int maze.search.MazeSolver.getVisitedCount () [inline]

Gets count of nodes visited in the current run.

Returns

number of visited nodes since last reset

9.18.3.7 isFinished()

boolean maze.search.MazeSolver.isFinished () [inline]

Indicates whether this run has terminated.

Returns

true when goal was found or search space was exhausted

9.18.3.8 isSolved()

boolean maze.search.MazeSolver.isSolved () [inline]

Indicates whether the current run found the goal.

Returns

true when goal node has been reached

9.18.3.9 resetSearch()

void maze.search.MazeSolver.resetSearch () [inline]

Resets transient search state so solving can start fresh.

This keeps the currently assigned maze and algorithm. Typical use is after changing visualization mode or when handling a restart.

9.18.3.10 setAlg()

void maze.search.MazeSolver.setAlg (
String alg) [inline]

Sets which algorithm to run.

Parameters

alg	algorithm label. Accepted values are DFS, BFS, UCS, Greedy, A∗, ASTAR, and A_STAR (case-insensitive)

Exceptions

IllegalArgumentException	if alg is null or unsupported

9.18.3.11 setMaze()

void maze.search.MazeSolver.setMaze (
Maze maze) [inline]

Assigns the maze instance to solve.

Parameters

maze	maze object to solve; start/goal are validated when solving begins

9.18.3.12 updateSolve()

Maze.Node maze.search.MazeSolver.updateSolve () [inline]

Advances solving by exactly one visited node.

Returns

node visited in this step, or null if solving is complete or no work remains

Exceptions

IllegalStateException	if maze/start/goal are not configured

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/search/MazeSolver.java

9.19 app.MS Class Reference

Demo application entry point.

Public Member Functions

- MS ()
  Creates the application entry-point holder.

Static Public Member Functions

- static void main (String[] args)
  Starts the Maze-Solver 0.8.1 application.

9.19.1 Detailed Description

Demo application entry point.

Typical usage is to run this class directly from IDE or command line. It delegates all GUI and session flow to ui.MazeRenderer.

9.19.2 Constructor & Destructor Documentation

9.19.2.1 MS()

app.MS.MS () [inline]

Creates the application entry-point holder.

9.19.3 Member Function Documentation

9.19.3.1 main()

void app.MS.main (
String[] args) [inline], [static]

Starts the Maze-Solver 0.8.1 application.

Delegates application flow to ui.MazeRenderer, including title screen mode selection, maze import or generation, and interactive solve controls.

Parameters

args	command line arguments (currently unused)

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/app/MS.java

9.20 maze.map.Maze.NeighborValidationResult Class Reference

Validation result for neighbor consistency checks.

Public Member Functions

- boolean isValid ()
  Indicates whether neighbor validation succeeded.
- int getIssueCount ()
  Gets the total number of detected issues.
- List< String > getIssues ()
  Gets the collected issue messages.

9.20.1 Detailed Description

Validation result for neighbor consistency checks.

9.20.2 Member Function Documentation

9.20.2.1 getIssueCount()

int maze.map.Maze.NeighborValidationResult.getIssueCount () [inline]

Gets the total number of detected issues.

Returns

issue count

9.20.2.2 getIssues()

List< String > maze.map.Maze.NeighborValidationResult.getIssues () [inline]

Gets the collected issue messages.

Returns

immutable list of issue descriptions

9.20.2.3 isValid()

boolean maze.map.Maze.NeighborValidationResult.isValid () [inline]

Indicates whether neighbor validation succeeded.

Returns

true when zero issues were found

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java

9.21 maze.map.Maze.Node Class Reference

Represents a single cell in the maze grid.

Public Member Functions

- Node (int row, int col)
  Creates a non-goal node at the given position.
- Node (int row, int col, boolean isGoal)
  Creates a node at the given position with an explicit goal flag.
- Node getNeighbor (Dir d)
  Retrieves the neighboring node in the given direction.
- void setNeighbor (Dir d, Node n)
  Assigns the neighboring node in the given direction.
- int getMoveCost (Dir d)
  Gets the movement cost in a direction.
- void setMoveCost (Dir d, int cost)
  Sets the movement cost in a direction.

Public Attributes

- final int row

  Zero-based row coordinate.
- final int col

  Zero-based column coordinate.
- boolean isGoal

  Whether this node is the maze goal.
- Node[] neighbors

  Adjacent nodes in [NORTH, EAST, SOUTH, WEST] order.

9.21.1 Detailed Description

Represents a single cell in the maze grid.

A null value in the neighbors array indicates either a wall or an out-of-bounds direction.

9.21.2 Constructor & Destructor Documentation

9.21.2.1 Node() [1/2]

```
maze.map.Maze.Node.Node (
            int row,
            int col)    [inline]
```

Creates a non-goal node at the given position.

Parameters

row	node row coordinate
col	node column coordinate

9.21.2.2 Node() [2/2]

```
maze.map.Maze.Node.Node (
            int row,
            int col,
            boolean isGoal)    [inline]
```

Creates a node at the given position with an explicit goal flag.

Parameters

row	node row coordinate
col	node column coordinate
isGoal	true if this node is a goal node, false otherwise

9.21.3 Member Function Documentation

9.21.3.1 getMoveCost()

int maze.map.Maze.Node.getMoveCost (
Dir d) [inline]

Gets the movement cost in a direction.

Parameters

d	direction to inspect

Returns

zero when movement is disabled, otherwise a cost in the range Maze#MIN_MOVEMENT_COST..Maze#MAX_MC

9.21.3.2 getNeighbor()

Node maze.map.Maze.Node.getNeighbor (
Dir d) [inline]

Retrieves the neighboring node in the given direction.

Parameters

d	direction to inspect

Returns

adjacent node, or null if blocked/out of bounds

9.21.3.3 setMoveCost()

void maze.map.Maze.Node.setMoveCost (
Dir d,
int cost) [inline]

Sets the movement cost in a direction.

Parameters

d	direction to update
cost	zero to disable movement, or a positive integer movement cost

Exceptions

IllegalArgumentException	if cost is outside 0..MAX_MOVEMENT_COST

9.21.3.4 setNeighbor()

void maze.map.Maze.Node.setNeighbor (
Dir d,
Node n) [inline]

Assigns the neighboring node in the given direction.

Parameters

d	direction to assign
n	adjacent node to store, or null

9.21.4 Member Data Documentation

9.21.4.1 col

final int maze.map.Maze.Node.col

Zero-based column coordinate.

9.21.4.2 isGoal

boolean maze.map.Maze.Node.isGoal

Whether this node is the maze goal.

9.21.4.3 neighbors

Node [] maze.map.Maze.Node.neighbors

Adjacent nodes in [NORTH, EAST, SOUTH, WEST] order.

9.21.4.4 row

final int maze.map.Maze.Node.row

Zero-based row coordinate.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java

9.22 experiments.ExperimentRunner.ProgressListener Interface Reference

Receives progress after each completed algorithm run.

Public Member Functions

- void onProgress (int completedRuns, int totalRuns, ExperimentResult result)
  Reports completed work.

9.22.1 Detailed Description

Receives progress after each completed algorithm run.

9.22.2 Member Function Documentation

9.22.2.1 onProgress()

void experiments.ExperimentRunner.ProgressListener.onProgress (
int completedRuns,
int totalRuns,
ExperimentResult result)

Reports completed work.

Parameters

completedRuns	completed algorithm runs
totalRuns	total planned algorithm runs
result	result that was just completed

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.23 experiments.TestSuite.ReachabilityMode Enum Reference

Whether generated maps must contain a route between their endpoints.

Public Attributes

- GUARANTEED_SOLVABLE
  Regenerates or repairs maps until a route exists.
- GUARANTEED_UNSOLVABLE
  Intentionally generates a map with no route between endpoints.
- NATURAL
  Leaves connectivity to the seeded random generation process.

9.23.1 Detailed Description

Whether generated maps must contain a route between their endpoints.

9.23.2 Member Data Documentation

9.23.2.1 GUARANTEED_SOLVABLE

experiments.TestSuite.ReachabilityMode.GUARANTEED_SOLVABLE

Regenerates or repairs maps until a route exists.

9.23.2.2 GUARANTEED_UNSOLVABLE

experiments.TestSuite.ReachabilityMode.GUARANTEED_UNSOLVABLE

Intentionally generates a map with no route between endpoints.

9.23.2.3 NATURAL

experiments.TestSuite.ReachabilityMode.NATURAL

Leaves connectivity to the seeded random generation process.

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.24 maze.search.SearchStrategy Interface Reference

Internal incremental search strategy used by MazeSolver.

Public Member Functions

- void initialize (MazeSolver solver)
- Maze.Node step (MazeSolver solver)

9.24.1 Detailed Description

Internal incremental search strategy used by MazeSolver.

The public solver remains the stable lifecycle and metrics facade while each strategy owns its frontier and algorithm-specific bookkeeping.

9.24.2 Member Function Documentation

9.24.2.1 initialize()

void maze.search.SearchStrategy.initialize (
MazeSolver solver)

9.24.2.2 step()

Maze.Node maze.search.SearchStrategy.step (
MazeSolver solver)

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/search/SearchStrategy.java

9.25 experiments.ExperimentRunner.SolveProgress Class Reference

Immutable diagnostic snapshot for one active solver worker.

Public Member Functions

- String getConfigurationId ()
  Gets the test-row configuration identifier.
- int getIteration ()
  Gets the one-based map iteration.
- MazeSolver.Algorithm getAlgorithm ()
  Gets the algorithm being measured.
- int getNodesExplored ()
  Gets the finalized nodes explored so far.
- long getPathCost ()
  Gets the current path cost.
- long getElapsedNs ()
  Gets elapsed solver time.
- boolean isFinished ()
  Indicates whether this snapshot represents the completed solve.

9.25.1 Detailed Description

Immutable diagnostic snapshot for one active solver worker.

9.25.2 Member Function Documentation

9.25.2.1 getAlgorithm()

MazeSolver.Algorithm experiments.ExperimentRunner.SolveProgress.getAlgorithm () [inline]

Gets the algorithm being measured.

Returns

algorithm being measured

9.25.2.2 getConfigurationId()

String experiments.ExperimentRunner.SolveProgress.getConfigurationId () [inline]

Gets the test-row configuration identifier.

Returns

test-row configuration identifier

9.25.2.3 getElapsedNs()

long experiments.ExperimentRunner.SolveProgress.getElapsedNs () [inline]

Gets elapsed solver time.

Returns

elapsed solver time in nanoseconds

9.25.2.4 getIteration()

int experiments.ExperimentRunner.SolveProgress.getIteration () [inline]

Gets the one-based map iteration.

Returns

one-based map iteration

9.25.2.5 getNodesExplored()

int experiments.ExperimentRunner.SolveProgress.getNodesExplored () [inline]

Gets the finalized nodes explored so far.

Returns

finalized nodes explored so far

9.25.2.6 getPathCost()

long experiments.ExperimentRunner.SolveProgress.getPathCost () [inline]

Gets the current path cost.

Returns

current path cost, or -1 when unavailable

9.25.2.7 isFinished()

boolean experiments.ExperimentRunner.SolveProgress.isFinished () [inline]

Indicates whether this snapshot represents the completed solve.

Returns

true when the solve is complete

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.26 experiments.ExperimentRunner.SolveProgressListener Interface Reference

Receives throttled progress from solver workers while they are active.

Public Member Functions

- void onProgress (SolveProgress progress)
  Reports the current state of one solver.

9.26.1 Detailed Description

Receives throttled progress from solver workers while they are active.

9.26.2 Member Function Documentation

9.26.2.1 onProgress()

void experiments.ExperimentRunner.SolveProgressListener.onProgress (
SolveProgress progress)

Reports the current state of one solver.

Parameters

progress	current worker state

The documentation for this interface was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.27 experiments.ExperimentRunner.TaskPriority Enum Reference

Public Member Functions

- TaskPriority (int weight)

Public Attributes

- MAP =(0)
- SOLVER =(1)

9.27.1 Constructor & Destructor Documentation

9.27.1.1 TaskPriority()

experiments.ExperimentRunner.TaskPriority.TaskPriority (
int weight) [inline]

9.27.2 Member Data Documentation

9.27.2.1 MAP

experiments.ExperimentRunner.TaskPriority.MAP =(0)

9.27.2.2 SOLVER

experiments.ExperimentRunner.TaskPriority.SOLVER =(1)

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/ExperimentRunner.java

9.28 experiments.TestSuite.TestCase Class Reference

Describes one row in a test suite.

Public Member Functions

- TestCase ()
  Creates a test case with proposal-oriented defaults.
- TestCase (String id)
  Creates a test case with an explicit identifier.
- String getId ()
  Returns the stable identifier for this test row.
- void setId (String id)
  Sets the stable row identifier.
- int getRows ()
  Returns the total row count, including the blocked outside border.
- void setRows (int rows)
  Sets the total row count, including the outside border.
- int getColumns ()
  Returns the total column count, including the blocked outside border.
- void setColumns (int columns)
  Sets the total column count, including the outside border.
- double getBlockedPercent ()
  Returns the requested blocked percentage among interior cells only.
- void setBlockedPercent (double blockedPercent)
  Sets the requested blocked percentage among interior cells.
- ReachabilityMode getReachabilityMode ()
  Returns the required connectivity policy for generated maps.
- void setReachabilityMode (ReachabilityMode reachabilityMode)
  Sets the required connectivity policy for generated maps.
- CostMode getCostMode ()
  Returns the movement-cost distribution mode.
- void setCostMode (CostMode costMode)
  Sets the movement-cost distribution mode.
- int getMinimumCost ()
  Returns the inclusive minimum generated movement cost.
- void setMinimumCost (int minimumCost)
  Sets the inclusive minimum generated movement cost.
- int getMaximumCost ()
  Returns the inclusive maximum generated movement cost.
- void setMaximumCost (int maximumCost)
  Sets the inclusive maximum generated movement cost.
- boolean isSameCostBothDirections ()
  Returns whether reverse directions share one generated cost.
- void setSameCostBothDirections (boolean sameCostBothDirections)
  Sets whether reverse directions share one generated cost.
- EndpointMode getEndpointMode ()
  Returns the endpoint-selection policy.
- void setEndpointMode (EndpointMode endpointMode)
  Sets the endpoint-selection policy.

- int getStartRow ()
  Returns the fixed start row when endpoint mode is FIXED.
- void setStartRow (int startRow)
  Sets the fixed start row.
- int getStartColumn ()
  Returns the fixed start column when endpoint mode is FIXED.
- void setStartColumn (int startColumn)
  Sets the fixed start column.
- int getGoalRow ()
  Returns the fixed goal row when endpoint mode is FIXED.
- void setGoalRow (int goalRow)
  Sets the fixed goal row.
- int getGoalColumn ()
  Returns the fixed goal column when endpoint mode is FIXED.
- void setGoalColumn (int goalColumn)
  Sets the fixed goal column.
- int getTargetManhattanDistance ()
  Returns the requested Manhattan distance for approximate endpoints.
- void setTargetManhattanDistance (int targetManhattanDistance)
  Sets the requested Manhattan distance for approximate endpoints.
- int getDistanceTolerance ()
  Returns the allowed absolute distance error.
- void setDistanceTolerance (int distanceTolerance)
  Sets the allowed absolute distance error.
- long getSeed ()
  Returns the persisted base seed.
- void setSeed (long seed)
  Sets an explicit seed and switches the row to manual seed mode.
- boolean isAutomaticSeed ()
  Returns whether the seed may be generated by the suite editor.
- void setAutomaticSeed (boolean automaticSeed)
  Sets whether the row uses automatic seed selection.
- MapSource getMapSource ()
  Returns the selected map source.
- void setMapSource (MapSource mapSource)
  Sets the selected map source.
- String getSavedMapText ()
  Returns the embedded saved map text.
- String getSavedMapName ()
  Returns the imported saved-map filename.
- void setSavedMapText (String savedMapText)
  Sets the complete embedded saved-map contents.
- void setSavedMapName (String savedMapName)
  Sets the imported saved-map filename.
- int getIterations ()
  Returns the shared iteration count for all selected algorithms.
- void setIterations (int iterations)
  Sets the shared iteration count for all selected algorithms.
- void setAlgorithms (Set< MazeSolver.Algorithm > algorithms)
  Replaces the selected algorithms.
- Set< MazeSolver.Algorithm > getAlgorithms ()
  Gets selected algorithms in insertion order.
- void validate ()
  Validates all values in this row.

9.28.1 Detailed Description

Describes one row in a test suite.

The setters intentionally allow temporary invalid values so a future editor can populate a row before calling validate(). A row must be validated before it is added to a suite or executed.

9.28.2 Constructor & Destructor Documentation

9.28.2.1 TestCase() [1/2]

experiments.TestSuite.TestCase.TestCase () [inline]

Creates a test case with proposal-oriented defaults.

9.28.2.2 TestCase() [2/2]

experiments.TestSuite.TestCase.TestCase (
String id) [inline]

Creates a test case with an explicit identifier.

Parameters

id	row identifier

9.28.3 Member Function Documentation

9.28.3.1 getAlgorithms()

Set< MazeSolver.Algorithm > experiments.TestSuite.TestCase.getAlgorithms () [inline]

Gets selected algorithms in insertion order.

Returns

unmodifiable algorithm set

9.28.3.2 getBlockedPercent()

double experiments.TestSuite.TestCase.getBlockedPercent () [inline]

Returns the requested blocked percentage among interior cells only.

Returns

interior blocked percentage

9.28.3.3 getColumns()

int experiments.TestSuite.TestCase.getColumns () [inline]

Returns the total column count, including the blocked outside border.

Returns

total column count

9.28.3.4 getCostMode()

CostMode experiments.TestSuite.TestCase.getCostMode () [inline]

Returns the movement-cost distribution mode.

Returns

movement-cost mode

9.28.3.5 getDistanceTolerance()

int experiments.TestSuite.TestCase.getDistanceTolerance () [inline]

Returns the allowed absolute distance error.

Returns

distance tolerance

9.28.3.6 getEndpointMode()

EndpointMode experiments.TestSuite.TestCase.getEndpointMode () [inline]

Returns the endpoint-selection policy.

Returns

endpoint-selection policy

9.28.3.7 getGoalColumn()

int experiments.TestSuite.TestCase.getGoalColumn () [inline]

Returns the fixed goal column when endpoint mode is FIXED.

Returns

fixed goal column

9.28.3.8 getGoalRow()

int experiments.TestSuite.TestCase.getGoalRow () [inline]

Returns the fixed goal row when endpoint mode is FIXED.

Returns

fixed goal row

9.28.3.9 getId()

String experiments.TestSuite.TestCase.getId () [inline]

Returns the stable identifier for this test row.

Returns

stable row identifier

9.28.3.10 getIterations()

int experiments.TestSuite.TestCase.getIterations () [inline]

Returns the shared iteration count for all selected algorithms.

Returns

iteration count

9.28.3.11 getMapSource()

MapSource experiments.TestSuite.TestCase.getMapSource () [inline]

Returns the selected map source.

Returns

selected map source

9.28.3.12 getMaximumCost()

int experiments.TestSuite.TestCase.getMaximumCost () [inline]

Returns the inclusive maximum generated movement cost.

Returns

maximum movement cost

9.28.3.13 getMinimumCost()

int experiments.TestSuite.TestCase.getMinimumCost () [inline]

Returns the inclusive minimum generated movement cost.

Returns

minimum movement cost

9.28.3.14 getReachabilityMode()

ReachabilityMode experiments.TestSuite.TestCase.getReachabilityMode () [inline]

Returns the required connectivity policy for generated maps.

Returns

connectivity policy

9.28.3.15 getRows()

int experiments.TestSuite.TestCase.getRows () [inline]

Returns the total row count, including the blocked outside border.

Returns

total row count

9.28.3.16 getSavedMapName()

String experiments.TestSuite.TestCase.getSavedMapName () [inline]

Returns the imported saved-map filename.

Returns

saved-map filename, or null when unavailable

9.28.3.17 getSavedMapText()

String experiments.TestSuite.TestCase.getSavedMapText () [inline]

Returns the embedded saved map text.

Returns

saved map text, or null when not selected

9.28.3.18 getSeed()

long experiments.TestSuite.TestCase.getSeed () [inline]

Returns the persisted base seed.

Returns

base seed

9.28.3.19 getStartColumn()

int experiments.TestSuite.TestCase.getStartColumn () [inline]

Returns the fixed start column when endpoint mode is FIXED.

Returns

fixed start column

9.28.3.20 getStartRow()

int experiments.TestSuite.TestCase.getStartRow () [inline]

Returns the fixed start row when endpoint mode is FIXED.

Returns

fixed start row

9.28.3.21 getTargetManhattanDistance()

int experiments.TestSuite.TestCase.getTargetManhattanDistance () [inline]

Returns the requested Manhattan distance for approximate endpoints.

Returns

target Manhattan distance

9.28.3.22 isAutomaticSeed()

boolean experiments.TestSuite.TestCase.isAutomaticSeed () [inline]

Returns whether the seed may be generated by the suite editor.

Returns

whether automatic seed selection is enabled

9.28.3.23 isSameCostBothDirections()

boolean experiments.TestSuite.TestCase.isSameCostBothDirections () [inline]

Returns whether reverse directions share one generated cost.

Returns

whether reverse directions share a cost

9.28.3.24 setAlgorithms()

void experiments.TestSuite.TestCase.setAlgorithms (
Set< MazeSolver.Algorithm > algorithms) [inline]

Replaces the selected algorithms.

Parameters

algorithms	non-empty algorithm collection

9.28.3.25 setAutomaticSeed()

void experiments.TestSuite.TestCase.setAutomaticSeed (
boolean automaticSeed) [inline]

Sets whether the row uses automatic seed selection.

Parameters

automaticSeed	automatic seed setting

9.28.3.26 setBlockedPercent()

void experiments.TestSuite.TestCase.setBlockedPercent (
double blockedPercent) [inline]

Sets the requested blocked percentage among interior cells.

Parameters

blockedPercent	interior blocked percentage

9.28.3.27 setColumns()

void experiments.TestSuite.TestCase.setColumns (
int columns) [inline]

Sets the total column count, including the outside border.

Parameters

columns	total column count

9.28.3.28 setCostMode()

void experiments.TestSuite.TestCase.setCostMode (
CostMode costMode) [inline]

Sets the movement-cost distribution mode.

Parameters

costMode	movement-cost mode

9.28.3.29 setDistanceTolerance()

void experiments.TestSuite.TestCase.setDistanceTolerance (
int distanceTolerance) [inline]

Sets the allowed absolute distance error.

Parameters

distanceTolerance	distance tolerance

9.28.3.30 setEndpointMode()

void experiments.TestSuite.TestCase.setEndpointMode (
EndpointMode endpointMode) [inline]

Sets the endpoint-selection policy.

Parameters

endpointMode	endpoint-selection policy

9.28.3.31 setGoalColumn()

void experiments.TestSuite.TestCase.setGoalColumn (
int goalColumn) [inline]

Sets the fixed goal column.

Parameters

goalColumn	fixed goal column

9.28.3.32 setGoalRow()

void experiments.TestSuite.TestCase.setGoalRow (
int goalRow) [inline]

Sets the fixed goal row.

Parameters

goalRow	fixed goal row

9.28.3.33 setId()

void experiments.TestSuite.TestCase.setId (
String id) [inline]

Sets the stable row identifier.

Parameters

id	non-blank identifier

9.28.3.34 setIterations()

void experiments.TestSuite.TestCase.setIterations (
int iterations) [inline]

Sets the shared iteration count for all selected algorithms.

Parameters

iterations	iteration count

9.28.3.35 setMapSource()

void experiments.TestSuite.TestCase.setMapSource (
MapSource mapSource) [inline]

Sets the selected map source.

Parameters

mapSource	selected map source

9.28.3.36 setMaximumCost()

void experiments.TestSuite.TestCase.setMaximumCost (
int maximumCost) [inline]

Sets the inclusive maximum generated movement cost.

Parameters

maximumCost	maximum movement cost

9.28.3.37 setMinimumCost()

void experiments.TestSuite.TestCase.setMinimumCost (
int minimumCost) [inline]

Sets the inclusive minimum generated movement cost.

Parameters

minimumCost	minimum movement cost

9.28.3.38 setReachabilityMode()

void experiments.TestSuite.TestCase.setReachabilityMode (
ReachabilityMode reachabilityMode) [inline]

Sets the required connectivity policy for generated maps.

Parameters

reachabilityMode	connectivity policy

9.28.3.39 setRows()

void experiments.TestSuite.TestCase.setRows (
int rows) [inline]

Sets the total row count, including the outside border.

Parameters

rows	total row count

9.28.3.40 setSameCostBothDirections()

void experiments.TestSuite.TestCase.setSameCostBothDirections (
boolean sameCostBothDirections) [inline]

Sets whether reverse directions share one generated cost.

Parameters

sameCostBothDirections	reverse-direction cost policy

9.28.3.41 setSavedMapName()

void experiments.TestSuite.TestCase.setSavedMapName (
String savedMapName) [inline]

Sets the imported saved-map filename.

Parameters

savedMapName	imported saved-map filename

9.28.3.42 setSavedMapText()

void experiments.TestSuite.TestCase.setSavedMapText (
String savedMapText) [inline]

Sets the complete embedded saved-map contents.

Parameters

savedMapText	complete map file contents

9.28.3.43 setSeed()

void experiments.TestSuite.TestCase.setSeed (
long seed) [inline]

Sets an explicit seed and switches the row to manual seed mode.

Parameters

seed	base seed for deterministic generation

9.28.3.44 setStartColumn()

void experiments.TestSuite.TestCase.setStartColumn (
int startColumn) [inline]

Sets the fixed start column.

Parameters

startColumn	fixed start column

9.28.3.45 setStartRow()

void experiments.TestSuite.TestCase.setStartRow (
int startRow) [inline]

Sets the fixed start row.

Parameters

startRow	fixed start row

9.28.3.46 setTargetManhattanDistance()

void experiments.TestSuite.TestCase.setTargetManhattanDistance (
int targetManhattanDistance) [inline]

Sets the requested Manhattan distance for approximate endpoints.

Parameters

targetManhattanDistance	target Manhattan distance

9.28.3.47 validate()

void experiments.TestSuite.TestCase.validate () [inline]

Validates all values in this row.

Exceptions

IllegalArgumentException	if a value is invalid

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.29 experiments.TestSuite Class Reference

Configuration model for repeatable maze-solving experiments.

Classes

- enum ExecutionMode
  Execution strategy for experiment solver runs.
- enum MapSource
  Source used to obtain the map for an experiment row.
- enum CostMode
  Movement-cost distribution used by an experiment row.
- enum ReachabilityMode
  Whether generated maps must contain a route between their endpoints.
- enum EndpointMode
  How an experiment row chooses its start and goal coordinates.
- class TestCase
  Describes one row in a test suite.

Public Member Functions

- TestSuite (String name)
  Creates an empty suite with default export settings.
- String getName ()
  Gets the suite name.
- void setName (String name)
  Sets the suite name.
- void addTestCase (TestCase testCase)
  Adds a validated test case to the suite.
- TestCase removeTestCase (int index)
  Removes a test case by index.
- List< TestCase > getTestCases ()
  Gets the suite rows as an unmodifiable view.
- boolean isAutomaticExport ()

Indicates whether results should be exported automatically after a run.

- void setAutomaticExport (boolean automaticExport)

  Enables or disables automatic result export.
- String getExportDirectory ()

  Gets the configured automatic-export directory.
- void setExportDirectory (String exportDirectory)

  Sets the automatic-export directory.
- ExecutionMode getExecutionMode ()

  Indicates whether independent algorithm runs may execute concurrently.
- void setExecutionMode (ExecutionMode executionMode)

  Sets the execution strategy for this suite.
- boolean isParallelExecution ()

  Compatibility accessor for callers using the original boolean option.
- void setParallelExecution (boolean parallelExecution)

  Compatibility setter for the original boolean option.
- void validate ()

  Validates the complete suite.

9.29.1 Detailed Description

Configuration model for repeatable maze-solving experiments.

A suite contains ordered test rows. Each row describes one condition and uses the same iteration count for every selected algorithm. Generation and execution are intentionally handled by ExperimentRunner, not by this configuration class.

9.29.2 Constructor & Destructor Documentation

9.29.2.1 TestSuite()

experiments.TestSuite.TestSuite (
String name) [inline]

Creates an empty suite with default export settings.

Parameters

name	suite name

9.29.3 Member Function Documentation

9.29.3.1 addTestCase()

void experiments.TestSuite.addTestCase (
TestCase testCase) [inline]

Adds a validated test case to the suite.

Parameters

test↩ Case	row to add

9.29.3.2 getExecutionMode()

ExecutionMode experiments.TestSuite.getExecutionMode () [inline]

Indicates whether independent algorithm runs may execute concurrently.

The default is false so existing suites retain sequential execution.

Returns

the configured execution strategy

9.29.3.3 getExportDirectory()

String experiments.TestSuite.getExportDirectory () [inline]

Gets the configured automatic-export directory.

Returns

directory path, or null when the default should be selected

9.29.3.4 getName()

String experiments.TestSuite.getName () [inline]

Gets the suite name.

Returns

suite name

9.29.3.5 getTestCases()

List< TestCase > experiments.TestSuite.getTestCases () [inline]

Gets the suite rows as an unmodifiable view.

Returns

ordered test rows

9.29.3.6 isAutomaticExport()

boolean experiments.TestSuite.isAutomaticExport () [inline]

Indicates whether results should be exported automatically after a run.

Returns

true when automatic export is enabled

9.29.3.7 isParallelExecution()

boolean experiments.TestSuite.isParallelExecution () [inline]

Compatibility accessor for callers using the original boolean option.

Returns

true for either parallel execution mode

9.29.3.8 removeTestCase()

TestCase experiments.TestSuite.removeTestCase (
int index) [inline]

Removes a test case by index.

Parameters

index	zero-based row index

Returns

removed test case

9.29.3.9 setAutomaticExport()

void experiments.TestSuite.setAutomaticExport (
boolean automaticExport) [inline]

Enables or disables automatic result export.

Parameters

automaticExport	export setting

9.29.3.10 setExecutionMode()

void experiments.TestSuite.setExecutionMode (
ExecutionMode executionMode) [inline]

Sets the execution strategy for this suite.

Parameters

executionMode	execution mode

9.29.3.11 setExportDirectory()

void experiments.TestSuite.setExportDirectory (
String exportDirectory) [inline]

Sets the automatic-export directory.

Parameters

exportDirectory	directory path, or null to use the default

9.29.3.12 setName()

void experiments.TestSuite.setName (
String name) [inline]

Sets the suite name.

Parameters

name	non-blank suite name

Exceptions

IllegalArgumentException	if the name is blank

9.29.3.13 setParallelExecution()

void experiments.TestSuite.setParallelExecution (
boolean parallelExecution) [inline]

Compatibility setter for the original boolean option.

Parameters

parallelExecution	whether fixed parallel execution should be used

9.29.3.14 validate()

void experiments.TestSuite.validate () [inline]

Validates the complete suite.

Exceptions

IllegalArgumentException	if the suite has no valid test rows

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java

9.30 experiments.TestSuiteIO Class Reference

Persistence and reporting utilities for experiment suites and results.

Static Public Member Functions

- static void save (TestSuite suite, Path file) throws IOException
  Saves a suite and materializes automatic seeds for reproducibility.
- static TestSuite load (Path file) throws IOException
  Loads and validates a previously saved suite.
- static void exportCsv (List< ExperimentResult > results, Path file) throws IOException
  Exports results to a UTF-8 CSV file.
- static Path defaultExportDirectory ()
  Returns the current application directory for exports and file chooser defaults.

Static Public Attributes

- static final String SUITE_EXTENSION = ".mstests"
  Extension used for editable experiment suite files.
- static final String SUITE_FORMAT_VERSION = "1"
  Current serialized suite schema version.

9.30.1 Detailed Description

Persistence and reporting utilities for experiment suites and results.

Suite files are versioned UTF-8 properties files with the .mstests extension. Result CSV files use RFC 4180-style quoting so suite names, identifiers, and paths remain valid when they contain commas or line breaks.

9.30.2 Member Function Documentation

9.30.2.1 defaultExportDirectory()

Path experiments.TestSuiteIO.defaultExportDirectory () [inline], [static]

Returns the current application directory for exports and file chooser defaults.

Returns

the current application directory

9.30.2.2 exportCsv()

void experiments.TestSuiteIO.exportCsv (
List< ExperimentResult > results,
Path file) throws IOException [inline], [static]

Exports results to a UTF-8 CSV file.

Parameters

results	completed results
file	destination CSV file

Exceptions

IOException	if the file cannot be written

9.30.2.3 load()

TestSuite experiments.TestSuiteIO.load (
Path file) throws IOException [inline], [static]

Loads and validates a previously saved suite.

Parameters

file	suite file

Returns

loaded suite

Exceptions

IOException	if the file cannot be read or is malformed

9.30.2.4 save()

void experiments.TestSuiteIO.save (
TestSuite suite,
Path file) throws IOException [inline], [static]

Saves a suite and materializes automatic seeds for reproducibility.

Parameters

suite	suite to save
file	destination file

Exceptions

IOException	if the file cannot be written

9.30.3 Member Data Documentation

9.30.3.1 SUITE_EXTENSION

final String experiments.TestSuiteIO.SUITE_EXTENSION = ".mstests" [static]

Extension used for editable experiment suite files.

9.30.3.2 SUITE_FORMAT_VERSION

final String experiments.TestSuiteIO.SUITE_FORMAT_VERSION = "1" [static]

Current serialized suite schema version.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuiteIO.java

9.31 ui.TestSuiteWindow Class Reference

Swing editor and execution window for configurable experiment suites.

Static Public Member Functions

- static void showWindow ()
  Opens a new test-suite window on the Swing event thread.
- static void showWindow (Runnable onBack)
  Opens the test-suite window and invokes the callback when the user navigates back to the previous screen.
- static void closeWindow ()
  Cancels any active experiment and closes the test-suite window.

9.31.1 Detailed Description

Swing editor and execution window for configurable experiment suites.

The table intentionally keeps one row equal to one TestSuite.TestCase. Experiment generation and solving remain in ExperimentRunner; this class only translates editor values and presents progress/results.

9.31.2 Member Function Documentation

9.31.2.1 closeWindow()

void ui.TestSuiteWindow.closeWindow () [inline], [static]

Cancels any active experiment and closes the test-suite window.

This method is safe to call during application-wide shutdown.

9.31.2.2 showWindow() [1/2]

void ui.TestSuiteWindow.showWindow () [inline], [static]

Opens a new test-suite window on the Swing event thread.

9.31.2.3 showWindow() [2/2]

void ui.TestSuiteWindow.showWindow (
Runnable onBack) [inline], [static]

Opens the test-suite window and invokes the callback when the user navigates back to the previous screen.

Parameters

onBack	callback invoked after this window is closed

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteWindow.java

9.32 maze.search.UninformedSearch.TraversalMode Enum Reference

Public Attributes

- DFS
- BFS

9.32.1 Member Data Documentation

9.32.1.1 BFS

maze.search.UninformedSearch.TraversalMode.BFS

9.32.1.2 DFS

maze.search.UninformedSearch.TraversalMode.DFS

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/maze/search/UninformedSearch.java

9.33 app.Version Class Reference

Release metadata for the Maze-Solver application.

Static Public Attributes

- static final String RELEASE = "0.8.1"
  Current application release version.

9.33.1 Detailed Description

Release metadata for the Maze-Solver application.

9.33.2 Member Data Documentation

9.33.2.1 RELEASE

final String app.Version.RELEASE = ”0.8.1” [static]

Current application release version.

The documentation for this class was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/app/Version.java

9.34 ui.MazeRenderer.WeightDisplayState Enum Reference

Describes how a weight label should be styled based on the solve state.

Public Attributes

- NORMAL
- TRAVERSED
- FINAL_PATH

9.34.1 Detailed Description

Describes how a weight label should be styled based on the solve state.

9.34.2 Member Data Documentation

9.34.2.1 FINAL_PATH

ui.MazeRenderer.WeightDisplayState.FINAL_PATH

9.34.2.2 NORMAL

ui.MazeRenderer.WeightDisplayState.NORMAL

9.34.2.3 TRAVERSED

ui.MazeRenderer.WeightDisplayState.TRAVERSED

The documentation for this enum was generated from the following file:

- C:/Users/ramse/Github/Maze-Solver/src/ui/MazeRenderer.java

Chapter 10

File Documentation

10.1 C:/Users/ramse/Github/Maze-Solver/README.md File Reference

10.2 C:/Users/ramse/Github/Maze-Solver/src/app/Application↩ Preferences.java File Reference

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Properties;
```

Classes

- class app.ApplicationPreferences

  Stores user-level application preferences that are needed before the JVM starts, such as the requested maximum heap size.

Packages

- package app

10.3 C:/Users/ramse/Github/Maze-Solver/src/app/MS.java File Reference

```
import javax.swing.SwingUtilities;
import ui.MazeRenderer;
```

Classes

- class app.MS

  Demo application entry point.

Packages

- package app

10.4 C:/Users/ramse/Github/Maze-Solver/src/app/Version.java File Reference

Classes

- class app.Version

  Release metadata for the Maze-Solver application.

Packages

- package app

10.5 C:/Users/ramse/Github/Maze-Solver/src/experiments/↩ ExperimentCli.java File Reference

```
import java.io.IOException;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;
```

Classes

- class experiments.ExperimentCli

  Headless command-line entry point for running and benchmarking experiment suites without starting Swing.

Packages

- package experiments

10.6 C:/Users/ramse/Github/Maze-Solver/src/experiments/↩ ExperimentResult.java File Reference

```
import maze.search.MazeSolver;
import maze.map.Maze;
```

Classes

- class experiments.ExperimentResult

  Immutable measurements from one algorithm run on one generated map.

Packages

- package experiments

10.7 C:/Users/ramse/Github/Maze-Solver/src/experiments/↩ ExperimentRunner.java File Reference

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.SecureRandom;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Future;
import java.util.concurrent.FutureTask;
import java.util.concurrent.PriorityBlockingQueue;
import java.util.concurrent.Semaphore;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.lang.management.ManagementFactory;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import com.sun.management.OperatingSystemMXBean;
import maze.map.Maze;
import maze.search.MazeSolver;
```

Classes

- class experiments.ExperimentRunner
  Executes configured experiment rows without Swing or rendering overhead.
- interface experiments.ExperimentRunner.ProgressListener
  Receives progress after each completed algorithm run.
- interface experiments.ExperimentRunner.MapProgressListener
  Receives progress after each map has finished generating.
- interface experiments.ExperimentRunner.SolveProgressListener
  Receives throttled progress from solver workers while they are active.
- class experiments.ExperimentRunner.SolveProgress
  Immutable diagnostic snapshot for one active solver worker.
- interface experiments.ExperimentRunner.CancellationToken
  Allows a caller to stop a long-running experiment.
- interface experiments.ExperimentRunner.ExecutionControl
  Adds cooperative pausing to cancellation for interactive runs.
- enum experiments.ExperimentRunner.TaskPriority

Packages

- package experiments

10.8 C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuite.java File Reference

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
import maze.search.MazeSolver;
import maze.map.Maze;
```

Classes

- class experiments.TestSuite
  Configuration model for repeatable maze-solving experiments.
- enum experiments.TestSuite.ExecutionMode
  Execution strategy for experiment solver runs.
- enum experiments.TestSuite.MapSource
  Source used to obtain the map for an experiment row.
- enum experiments.TestSuite.CostMode
  Movement-cost distribution used by an experiment row.
- enum experiments.TestSuite.ReachabilityMode
  Whether generated maps must contain a route between their endpoints.
- enum experiments.TestSuite.EndpointMode
  How an experiment row chooses its start and goal coordinates.
- class experiments.TestSuite.TestCase
  Describes one row in a test suite.

Packages

- package experiments

10.9 C:/Users/ramse/Github/Maze-Solver/src/experiments/TestSuiteIO.java File Reference

```
import java.io.IOException;
import java.io.Reader;
import java.io.Writer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.List;
import java.util.Properties;
import java.util.Set;
import java.util.StringJoiner;
import app.Version;
import maze.search.MazeSolver;
```

Classes

- class experiments.TestSuiteIO

  Persistence and reporting utilities for experiment suites and results.

Packages

- package experiments

10.10 C:/Users/ramse/Github/Maze-Solver/src/maze/map/Maze.java File Reference

```
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;
import experiments.TestSuite;
```

Classes

- class maze.map.Maze

  Core maze model storing occupancy, neighbor links, and start/goal nodes.
- enum maze.map.Maze.Dir

  Cardinal directions used for indexing node neighbors.
- class maze.map.Maze.Node

  Represents a single cell in the maze grid.
- class maze.map.Maze.GeneratedExperimentMap

  Metadata for one generated experiment map.
- class maze.map.Maze.NeighborValidationResult

  Validation result for neighbor consistency checks.

Packages

- package maze.map

10.11 C:/Users/ramse/Github/Maze-Solver/src/maze/map/MazeFileIO.java File Reference

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;

Classes

- class maze.map.MazeFileIO

  Reads and writes the human-readable .msmap format.

Packages

- package maze.map

10.12 C:/Users/ramse/Github/Maze-Solver/src/maze/search/Astar.java File Reference

import java.util.PriorityQueue;
import maze.map.Maze;

Classes

- class maze.search.Astar

  Incremental weighted A∗ strategy using Manhattan distance as its heuristic.

Packages

- package maze.search

10.13 C:/Users/ramse/Github/Maze-Solver/src/maze/search/GreedySearch.java File Reference

import java.util.PriorityQueue;
import maze.map.Maze;

Classes

- class maze.search.GreedySearch

  Greedy Best-First Search strategy using Manhattan distance as its priority.

Packages

- package maze.search

10.14 C:/Users/ramse/Github/Maze-Solver/src/maze/search/MazeSolver.java File Reference

import maze.map.Maze;
import java.awt.Point;
import java.util.ArrayList;
import java.util.List;

Classes

- class maze.search.MazeSolver

  Incremental maze solver that supports DFS, BFS, Uniform Cost Search, Greedy Best-First Search, and weighted A∗.
- enum maze.search.MazeSolver.Algorithm

  Search strategies supported by this solver.

Packages

- package maze.search

10.15 C:/Users/ramse/Github/Maze-Solver/src/maze/search/SearchStrategy.java File Reference

import maze.map.Maze;

Classes

- interface maze.search.SearchStrategy

  Internal incremental search strategy used by MazeSolver.

Packages

- package maze.search

10.16 C:/Users/ramse/Github/Maze-Solver/src/maze/search/UCS.java File Reference

import java.util.PriorityQueue;
import maze.map.Maze;

Classes

- class maze.search.UCS

  Uniform-Cost Search strategy.

Packages

- package maze.search

10.17 C:/Users/ramse/Github/Maze-Solver/src/maze/search/↩ UninformedSearch.java File Reference

import java.util.ArrayDeque;
import java.util.Deque;
import maze.map.Maze;

Classes

- class maze.search.UninformedSearch

  Incremental implementation for the uninformed DFS and BFS algorithms.
- enum maze.search.UninformedSearch.TraversalMode

Packages

- package maze.search

10.18 C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultsPanel.java File Reference

```
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.util.Objects;
import java.util.function.IntConsumer;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JCheckBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JProgressBar;
import javax.swing.JScrollPane;
import javax.swing.JSplitPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.JTextArea;
```

Classes

- class ui.ExperimentResultsPanel

  Layout and actions for experiment execution controls and results.

Packages

- package ui

10.19 C:/Users/ramse/Github/Maze-Solver/src/ui/ExperimentResultTableModel.java File Reference

```
import javax.swing.table.DefaultTableModel;
```

Classes

- class ui.ExperimentResultTableModel

  Read-only table model for measured experiment results.

Packages

- package ui

10.20 C:/Users/ramse/Github/Maze-Solver/src/ui/MazeRenderer.java File Reference

```
import app.ApplicationPreferences;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.FileDialog;
import java.awt.FlowLayout;
import java.awt.Frame;
import java.awt.Font;
import java.awt.GradientPaint;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.GraphicsEnvironment;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.Point;
import java.awt.Rectangle;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.IOException;
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSpinner;
import javax.swing.JViewport;
import javax.swing.ScrollPaneConstants;
import javax.swing.SpinnerNumberModel;
import javax.swing.SwingUtilities;
import javax.swing.Timer;
import maze.map.Maze;
import maze.search.MazeSolver;
```

Classes

- class ui.MazeRenderer

  Renderer and UI controller for the maze application.
- enum ui.MazeRenderer.WeightDisplayState

  Describes how a weight label should be styled based on the solve state.

Packages

- package ui

10.21 C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfiguration↩ Table.java File Reference

```
import java.awt.Component;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.ListSelectionModel;
import javax.swing.JMenuItem;
import javax.swing.JPopupMenu;
import javax.swing.JTable;
import javax.swing.table.JTableHeader;
import javax.swing.table.TableCellRenderer;
import java.util.function.IntConsumer;
import experiments.TestSuite;
```

Classes

- class ui.TestConfigurationTable

  Configuration table with context-sensitive editability and disabled-cell rendering for saved-map and endpoint modes.

Packages

- package ui

10.22 C:/Users/ramse/Github/Maze-Solver/src/ui/TestConfiguration↩ TableModel.java File Reference

```
import javax.swing.table.DefaultTableModel;
import java.util.HashSet;
import java.util.Set;
```

Classes

- class ui.TestConfigurationTableModel

  Table model for editable experiment configurations.

Packages

- package ui

10.23 C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteConfigurationPanel.java File Reference

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.util.Objects;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;

Classes

- class ui.TestSuiteConfigurationPanel

  Layout and actions for editing the suite name and test configurations.

Packages

- package ui

10.24 C:/Users/ramse/Github/Maze-Solver/src/ui/TestSuiteWindow.java File Reference

import java.awt.BorderLayout;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.concurrent.CancellationException;
import java.util.Map;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JProgressBar;
import javax.swing.JSplitPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingWorker;
import javax.swing.Timer;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.event.WindowAdapter;

```
import java.awt.event.WindowEvent;
import experiments.ExperimentResult;
import experiments.ExperimentRunner;
import experiments.TestSuite;
import experiments.TestSuiteIO;
import maze.map.Maze;
import maze.search.MazeSolver;
```

Classes

- class ui.TestSuiteWindow

  Swing editor and execution window for configurable experiment suites.

Packages

- package ui

10.25 C:/Users/ramse/Github/Maze-Solver/src/ui/UITheme.java File Reference

```
import java.awt.Color;
import java.awt.Component;
import java.awt.Container;
import java.awt.Font;
import java.awt.Window;
import javax.swing.AbstractButton;
import javax.swing.BorderFactory;
import javax.swing.JComboBox;
import javax.swing.Icon;
import javax.swing.JComponent;
import javax.swing.JCheckBox;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JSpinner;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.UIManager;
import javax.swing.SwingUtilities;
import javax.swing.DefaultListCellRenderer;
import java.util.ArrayList;
import java.util.List;
import javax.swing.JButton;
```

Classes

- class ui.UITheme

  Shared visual language for the Swing screens.

Packages

- package ui